

No Single Model Family Dominates: Ensembles and Conventional Machine Learning Perform Comparably to Pretrained Molecular Models Across 44 Property-Prediction Benchmarks

Scott Coffin^{1*}

^{1*}California Office of Environmental Health Hazard Assessment, 1001 I Street, Sacramento, 95814, CA, USA.

Corresponding author(s). E-mail(s): scott.l.coffin@gmail.com;

Abstract

Background. ADMET prediction is dominated by ever-larger pretrained models, whose compute cost and reproducibility limit adoption in academic, regulatory and small-laboratory settings. Whether that scale is warranted has not been tested across suites under one pipeline.

Implementation. QSARena predicts molecular properties from SMILES in one reproducible workflow: RDKit standardization, ten fingerprint and descriptor families, train-only ElasticNetCV selection, and a 28-model library spanning conventional machine learning, gradient boosting, deep tabular and graph networks, 3D pretrained Uni-Mol, and ensembles. It runs two ways from one shared core: a **code-free Google Colab notebook needing no installation and no local hardware**, and a command-line runner.

Results. Across 44 datasets from five suites (837 model-dataset evaluations) under a **single fixed configuration with no per-dataset tuning**, QSARena reached the estimated published top ten on 35 of 37 leaderboard-comparable datasets, median rank 3, including all 22 TDC datasets under official **admet.-group** splits; selecting by cross-validation alone reduces this to 25 of 37, median rank 8. A matched-candidate-set control attributes 7 of those 10 placements to model-library breadth and only 3 to honest selection, which nonetheless removes every estimated first place. No family dominated: ensembles won 16 datasets, 3D pretrained models 11 and conventional machine learning 8, none exceeding 36%. Ensembles were most consistent, within 5% of best on 75% of datasets against 70% for 3D pretrained models at $\sim 55\times$ the cost. Graph networks are under-represented: four of five Chemprop variants failed.

Conclusions. A GPU-optional AutoML pipeline delivers competitive ADMET prediction out of the box, without tuning, installation or specialized hardware; the best model family is dataset-dependent. Results are single-split, so close margins are provisional.

Scientific Contribution. We report the broadest cross-suite, leakage-controlled ADMET benchmark we are aware of: 44 datasets, 28 models, 837 evaluations, one fixed pipeline. We decompose apparent leaderboard standing into model-library breadth (7 of 37 placements) and selection on held-out data (3 of 37, plus every first place), an inflation affecting any comparably selected entry but rarely reported. We show this needs neither datacentre hardware nor expert iteration: a consumer GPU reproduces it to within 0.2%, and the pipeline runs code-free in a browser.

Keywords: QSAR, ADMET, AutoML, Molecular property prediction, Benchmarking, Reproducibility, Data leakage, Ensemble learning, Foundation models, Open-source software, Accessibility

QSARena: top-10 on 35 of 37 ADMET benchmarks

One leakage-controlled pipeline, 44 datasets, 28 models - and no model family dominates.

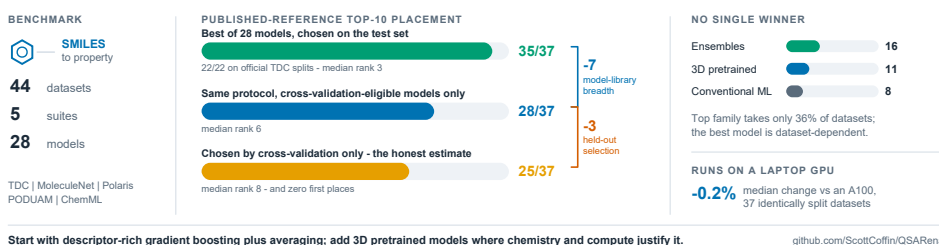


Fig. 1 *

Graphical abstract. QSARena evaluates conventional, ensemble and pretrained molecular models under one leakage-controlled pipeline across 44 benchmarks. No architecture family wins more than 36% of datasets; the best model per dataset reaches the estimated published top ten on 35 of 37 comparable datasets, falling to 25 of 37 when the model is chosen without consulting held-out data.

1 Background

Unfavourable absorption, distribution, metabolism, excretion and toxicity (ADMET) properties remain among the most consequential causes of failure in drug development. Approximately 90% of drug candidates that enter clinical testing fail across phases I–III and the subsequent approval process; the dominant causes are lack of clinical efficacy and unmanageable toxicity, with poor drug-like properties — including unfavourable pharmacokinetics — contributing a smaller but material share, having fallen from 30–40% of failures in the 1990s to 10–15% today [1]. The value of screening such liabilities early is well established: inappropriate pharmacokinetics and bioavailability accounted for roughly 40% of clinical attrition in the early 1990s, a share that fell to about 10% by 2000 as the industry adopted routine early ADME profiling [2]. Because experimental ADMET assays are slow, costly and hard to scale to the growing number of synthesized and virtual compounds, *in silico* prediction of ADMET endpoints from chemical structure has become an indispensable complement to laboratory screening [3, 4].

Standardized public benchmarks have driven much of this progress. MoleculeNet established curated datasets and evaluation protocols for molecular machine learning [5], and the Therapeutics Data Commons (TDC) consolidated a large collection of ADMET datasets into a benchmark group with fixed splits, per-task metrics and a public leaderboard enabling direct comparison [6]. More recent resources such as Polaris have emphasized immutable, standardized benchmarks as a route to reproducible comparison. These resources catalysed rapid methodological progress, but also fostered a leaderboard culture in which incremental gains are pursued through increasingly elaborate architectures.

A prominent expression of this trend is the turn toward large pretrained “foundation” models. Graph-based models such as MolE, pretrained on roughly 842 million molecules [7], and MolGPS, scaled to three billion parameters with the aid of phenomics data [8], have established state-of-the-art results on subsets of the TDC ADMET tasks, and parameter-efficient successors such as MiniMol have continued this line of work [9]. Three-dimensional pretrained models such as Uni-Mol, which learns from conformer geometry rather than 2D topology alone, represent a distinct and comparatively cheap point on this spectrum [10]. While impressive, the largest of these models require very large pretraining corpora, specialized hardware and substantial engineering expertise, placing them out of practical reach for many of the settings in which ADMET prediction is most needed.

Critically, the assumption that architectural scale translates into superior ADMET prediction is not well supported. In the most comprehensive benchmark of its kind, Xia et al. evaluated twelve representative models — three non-deep and nine deep — and found that deep models generally fail to outperform non-deep ones, with gradient-boosted trees and random forests on molecular fingerprints tending to perform best, because tree models suit the non-smooth target functions characteristic of molecular property prediction [11]. A review in the *Annual Review of Biomedical Data Science* reached a concordant conclusion: a substantial and consistent advantage of deep learning over standard machine learning across diverse datasets and properties has not been demonstrated, and success in compound-property prediction does not necessarily scale

with model complexity [12]. These observations are borne out on the TDC leaderboard itself, where gradient boosting on combined fingerprint and descriptor representations remains highly competitive: extreme gradient boosting (ADMETboost) [13]; CatBoost paired with ECFP, Avalon and ErG fingerprints plus ~ 200 molecular properties, which achieved top-3 performance on 16 of 22 benchmarks (the MapLight submission) [14]; AutoML over descriptor sets (CaliciBoost) [15]; and automatic feature-combination frameworks built on simple learners (MaxQsaring, ranked first on 19 of 22 TDC tasks) [16]. Systematic representation studies reinforce this: the choice of molecular representation is often more decisive than model architecture, and optimal choices are strongly dataset-dependent [17].

Compounding the questionable returns of architectural complexity is a deepening concern over reproducibility. Across the quantitative sciences, data leakage has been identified as a pervasive and often invisible cause of over-optimistic results, affecting at least 294 papers across 17 disciplines in one survey [18]. Cheminformatics is not exempt: leakage through structure duplication, preprocessing on combined train-test data, and feature selection performed before cross-validation systematically inflates QSAR performance estimates, and reproducibility, interpretability and generalizability deficits have hindered regulatory uptake of ML-based toxicity models [19]. A recent critical assessment of the TDC ADMET leaderboard found that only three top-ranked entries — CaliciBoost, MapLight and MapLight + GNN — passed all reproducibility checks, with most leading submissions exhibiting unavailable code, non-reproducible environments or methodological flaws [20]. Headline rankings therefore frequently fail to reflect either genuine methodological progress or deployable models.

A less-discussed form of the same problem concerns model *selection*. Benchmark suites report the performance of a chosen model on a held-out test set, but when the choice among many candidate models is itself made by comparing test-set scores, the reported number is an optimistic maximum over candidates rather than an estimate of what a practitioner would obtain on new data. This affects AutoML systems especially, since their value proposition is precisely that they search over many models. We treat it here as a first-class result rather than a caveat.

These problems are aggravated by a persistent accessibility gap. Many academic drug-discovery efforts founder in the preclinical “death valley” in part because researchers lack access to commercial ADME prediction software owing to high licensing fees [4]. Open tools have begun to address this, and the peer group is now substantial. ADMET-AI holds the highest average rank on the TDC ADMET leaderboard using a single Chemprop-RDKit architecture trained across 41 TDC datasets, served through a web interface and a command-line tool; its hosted service predicts from pretrained models, though the open-source release also ships training scripts [21]. Among AutoML frameworks, QSARtuna automates the comparison of molecular representations and learners with Optuna, with uncertainty quantification and explainability by design [22]; ZairaChem provides a fully automated, low-resource AutoML pipeline reported to reach state-of-the-art performance out of the box on the TDC ADMET binary classification tasks [23]; QSPRpred offers a flexible open QSPR toolkit with standardized serialization for reproducibility [24]; DeepMol delivered competitive, fully reproducible pipelines across 22 TDC ADMET datasets [25]; DeepPurpose

exposes 15 encoders and more than 50 architectures in a few lines of code, though it is drug-target-interaction-centric and used mainly as a TDC baseline [26]; and Auto-ADMET coupled grammar-based genetic programming with a Bayesian network to produce interpretable pipelines [27]. Code-light tools such as ChemXploreML have separately sought to lower the barrier for non-specialists through a desktop GUI, validated on physicochemical rather than ADMET endpoints [28].

Each individual capability of the present work therefore already exists somewhere in this landscape. What no single tool combines is a code-free interface, breadth across model families, uniform leakage-controlled evaluation across multiple benchmark suites, and feasibility on consumer hardware. That combination, rather than any one feature, is what we claim (Section 3.12).

Here we present QSARena, a portable QSAR modelling and benchmarking workspace designed to close these gaps. QSARena couples a code-free, GPU-optional notebook with a resume-safe command-line benchmark runner, both calling a shared workflow core that enforces train-only feature filtering and selection. The same pipeline spans conventional machine learning, gradient boosting, deep tabular and graph neural networks, 3D pretrained models, descriptor-graph hybrids, and combinatorial-fusion and stacking ensembles, and is evaluated uniformly across 44 datasets from five benchmark collections. We claim breadth on the cross-suite and model-family axes rather than on raw dataset count: ADMET-AI evaluates 41 TDC ADMET datasets and holds the highest average rank on that leaderboard [21], and DeepMol reports competitive pipelines across 22 [25], so the distinguishing features here are the number of suites and model families compared under one pipeline, combined with a code-free interface. Our contribution is therefore a unifying, leakage-controlled benchmarking framework and an architecture-agnostic empirical result, rather than a claim to leading accuracy. Using the framework we show that no single family dominates, that ensembles and conventional machine learning perform comparably to pretrained models on most benchmarks at a small fraction of the compute, and that the apparent margin between an AutoML system and a published leaderboard depends heavily on whether model selection is honest about held-out data. We are explicit in Section We are explicit in Section 3.12 that we do not claim accuracy leadership: several systems report stronger per-endpoint results, and against the leading open tool the comparison is untested rather than lost, because our ADMET-AI-equivalent backend ran on only a fraction of the suite. The finding, the breadth and the accessibility are the contribution. Equally important for practitioners, every result reported here was obtained under one fixed configuration with no per-dataset tuning, from a tool that can be run without installing anything: the accessibility and the accuracy are not in tension.

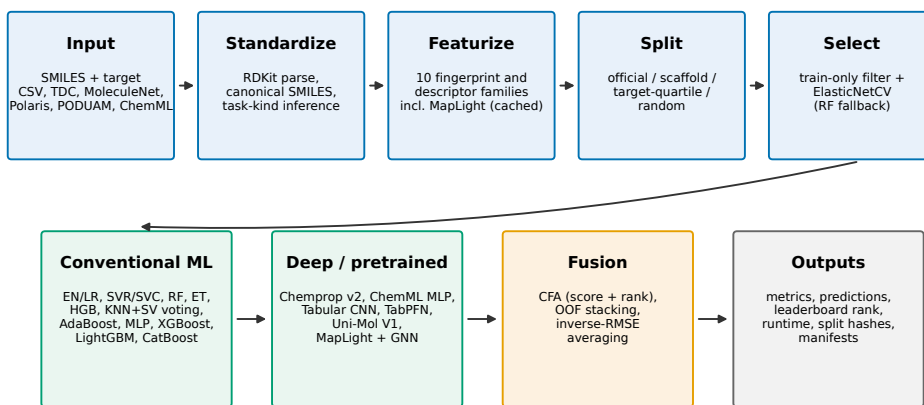
2 Implementation

2.1 Software architecture

QSARena is designed so that a user with a CSV of SMILES and measured values can obtain a benchmarked model without writing code, installing software or owning hardware. It predicts molecular properties directly from SMILES strings and is distributed with two interoperable entry points sharing a common workflow core:

`colab_qsar_tutorial.ipynb`, an interactive widget-driven Jupyter/Colab notebook for code-free model building on built-in or user-supplied data; and `run_qsarena-ga_benchmarks.py`, a command-line runner for resume-safe model comparison across curated dataset collections. Both call the same feature-generation, splitting, fusion and evaluation library (`qsar_workflow_core.py`), so interactive and batch results are produced by identical code paths (Fig. 2).

The notebook path is deliberately zero-friction: it is a single self-contained file that opens in Google Colab, installs its own dependencies in the hosted runtime, and exposes dataset choice, SMILES and target columns, split strategy and model selection through form widgets. The user edits no code. Because Colab supplies the compute, no local CPU, GPU or Python environment is required, which removes the installation and hardware barriers that limit uptake of ADMET tooling in the settings described in the Background. The command-line runner targets the opposite case — long, resumable, many-dataset campaigns on a workstation or HPC allocation — and both paths call identical code, so a result obtained in the browser can be reproduced in batch without change.



Notebook (code-free, Colab) and CLI runner share one workflow core; every stage is cached and resume-safe.

Fig. 2 The QSARena workflow. Data ingestion and standardization, featurization, splitting and train-only feature selection are followed by a model library spanning conventional machine learning, deep and pretrained models, and fusion; every stage writes cached, resumable artifacts. The code-free notebook and the command-line runner share this core.

For each dataset the runner executes a fixed sequence: build molecular features; apply the train/test split and train-only ElasticNetCV feature selection; evaluate conventional models; optionally run a genetic-algorithm tuning pass; run deep workflows (ChemML backends, Chemprop v2 variants, Uni-Mol V1, MapLight + GNN); optionally run combinatorial fusion over all successful predictions; build ensembles over available members; and write cross-dataset performance tables. Optional families are skipped gracefully when dependencies, hardware or dataset-size guardrails are

unmet, allowing the rest of a run to continue. The model inventory and its per-dataset availability are given in Table 1.

Table 1: Model inventory. “Valid” datasets are those on which the model produced a metric without an error; differences from 44 reflect task-type applicability (regression-only or classification-only estimators), dataset-size guardrails, or backend failures (Section 3.1).

Model family	Model	Valid regression datasets	Valid classification datasets	Wins	Datasets attempted
CFA combinatorial fusion	CFA (Combinatorial Fusion)	22	22	1	44
Chemprop v2 GNN	Chemprop v2 (AttentiveFP, ensemble=3)	2	4	1	44
Chemprop v2 GNN	Chemprop v2 (CMPNN, ensemble=3)	2	4	0	44
Chemprop v2 GNN	Chemprop v2 (D-MPNN + RDKit2D, ensemble=3)	2	4	1	44
Chemprop v2 GNN	Chemprop v2 (D-MPNN + Selected descriptors, ensemble=3)	2	4	0	44
Chemprop v2 GNN	Chemprop v2 (D-MPNN, ensemble=3)	2	4	0	44
Conventional ML	AdaBoost	22	22	2	44
Conventional ML	CatBoost	22	22	1	44
Conventional ML	ElasticNetCV	20	0	0	22
Conventional ML	Extra trees	22	22	0	44
Conventional ML	HistGradient Boosting	22	22	1	44
Conventional ML	LogisticRegression	0	22	0	22
Conventional ML	MapLight CatBoost (Strict Parity)	22	0	0	22
Conventional ML	Random forest	22	22	4	44
Conventional ML	SVC	0	22	0	22
Conventional ML	SVR	22	0	0	22
Conventional ML	Tabular CNN	22	0	0	22
Conventional ML	Tabular MLP	22	22	0	44
Conventional ML	Voting Classifier (KNN, SVM)	0	22	0	22
Conventional ML	Voting Regressor (KNN, SVM)	22	0	0	22

continued on next page

Model family	Model	Valid regression datasets	Valid classification datasets	Wins	Datasets attempted
Conventional ML	XGBoost	22	22	0	44
Deep tabular NN (ChemML MLP)	ChemML MLP (PyTorch)	22	22	1	44
Deep tabular NN (ChemML MLP)	ChemML MLP (TensorFlow)	22	22	1	44
Ensemble (stacking / averaging)	Ensemble (OOF Stacking (RidgeCV, 5-fold))	22	22	7	44
Ensemble (stacking / averaging)	Ensemble (Weighted average (inverse train RMSE))	22	22	9	44
MapLight + GNN	MapLight + GNN (CatBoost, Strict Parity)	22	22	4	44
Uni-Mol (3D pretrained)	Uni-Mol V1	22	22	7	44
Uni-Mol (3D pretrained)	Uni-Mol V2 (84m)	10	7	4	44

2.2 Datasets and curation

Benchmark datasets were drawn from five public collections through dedicated loaders: ChemML bundled examples (`cep_homo`, `organic_density`, `xyz_polarizability`); TDC single-prediction ADME and Tox tasks; MoleculeNet physicochemical datasets (ESOL [29], FreeSolv [30], Lipophilicity); Polaris ADME benchmark mirrors; and the PODUAM point-of-departure datasets [31]. Each dataset is represented internally by a `DatasetSpec` recording the SMILES column, target column, recommended split, recommended metric, benchmark suite and any leaderboard reference metadata.

All molecules were standardized before featurization. Canonicalization coerced the target to numeric, removed rows with missing or non-finite SMILES or target values, and parsed each SMILES with RDKit [32]; molecules that failed to parse were dropped and the remainder re-encoded as canonical SMILES. For datasets carrying a predefined split column, rows missing the split assignment were removed.

Target transformation followed a suite-aware policy. Under the default `auto` mode, datasets from the TDC, MoleculeNet, Polaris, literature and PFAS auxiliary suites were kept on their native scale, while other datasets used a base-10 logarithmic transform when all target values were positive; non-positive targets disabled the transform.

For TDC datasets, official splits were preferred over generic resampling. Where PyTDC exposes an official `admet_group` entry, the runner uses that `train_val/test` split in preference to legacy single-prediction cache entries; official split frames are cached under `data/_qsarena_cache` and stale entries lacking an official split are refreshed

automatically. This distinction matters for interpretation and we preserve it throughout: 22 TDC datasets and 5 Polaris datasets used official predefined splits, while the remaining 18 used locally generated scaffold, random or target-quartile splits and are therefore *not* directly leaderboard-equivalent (Section 3.4).

Task kind was inferred automatically rather than taken from catalog metadata: strict binary 0/1 targets were routed to the classification workflow and all other numeric targets to regression. This inference overrides several catalog entries that record a regression metric for what are in fact binary endpoints (for example the CYP inhibition tasks), and the inferred kind is what all analyses below use.

2.3 Molecular representations

Ten configurable feature families were implemented on RDKit, all returning fixed-width numeric matrices. Circular and path-based fingerprints were generated at 1024 bits by default. Morgan fingerprints [33] used radius 2; ECFP6 and FCFP6 were generated with the RDKit MorganGenerator at radius 3, FCFP6 using feature-based atom invariants; RDKit layered, atom-pair, topological-torsion and RDKit path fingerprints were each computed at 1024 bits, the path fingerprint spanning path lengths 1–7. MACCS keys [34] were generated at their native 167-bit width, and the full RDKit 2D descriptor set was computed from `Descriptors._descList`.

The “MapLight classic” family reproduces the feature construction of the MapLight TDC submission [14]: a hashed Morgan count fingerprint (radius 2, 1024 bits), an Avalon count fingerprint (1024 bits) [35], the extended reduced graph (ErG) fingerprint [36], and a curated panel of approximately 200 RDKit descriptors spanning connectivity and shape indices (BalabanJ, BertzCT, Chi and Kappa indices), VSA-family descriptors, physicochemical properties (ExactMolWt, MolLogP, TPSA, FractionCSP3), hydrogen-bonding and ring counts, functional-group fragment counts, and the QED drug-likeness score. In the feature-selection analyses below, this composite appears as its three constituent families (`avalon`, `erg` and the `maplight` descriptor panel), and we report both the components and their sum.

After assembly, every feature matrix passed a common finalization step: infinities replaced with missing values, all columns coerced to numeric, values exceeding the float32 range masked, remaining gaps imputed with the per-column median (falling back to zero), and the matrix cast to float32.

2.4 Persistent feature store

To avoid recomputing expensive descriptors across datasets and runs, features were cached in a persistent, content-addressed store writing Parquet shards where `pyarrow` or `fastparquet` is available and CSV otherwise. Each representation is identified by a key derived from a SHA-256 hash of a canonical JSON payload listing the selected feature families, Morgan radius and fingerprint bit width, so distinct configurations never collide. A schema file records the representation payload and column order; a mismatch raises an error rather than silently mixing definitions. Feature rows are keyed by canonical SMILES, so only molecules absent from the cache are computed on a given run, and cached and new rows are merged back into the exact requested

row order. Additional caches store assembled benchmark feature matrices and tuned conventional models.

2.5 Splitting and cross-validation

Four splitting strategies were supported — random, target-quartile, scaffold and predefined — selected per dataset with a benchmark-aware default. Command-line defaults set target-quartile stratification, a test fraction of 0.2 and random seed 13.

Scaffold splitting used Bemis–Murcko scaffolds [37] computed after removing stereochemistry, with explicit fallbacks for molecules that failed scaffold perception or yielded no scaffold. Molecules were grouped by scaffold and assigned to the test set by a greedy, seeded procedure that packs whole scaffold groups until the target test fraction is reached, guaranteeing that no scaffold appears in both partitions. Target-quartile splitting binned the continuous target into quartiles with `pd.qcut` and stratified on those bins, reverting to random splitting when a dataset was too small or too low in target variance to form at least two populated bins per fold. For TDC benchmark datasets the official partition was used directly, with deterministic reassignment by SMILES so that the same molecule is never placed in both train and test.

Cross-validation mirrored the split geometry: scaffold runs used `GroupKFold` over scaffold groups, target-quartile runs `StratifiedKFold` over quartile bins, and random runs shuffled `KFold`, each with a configurable fold count and a defined fallback to random folds when group or bin constraints could not be satisfied.

2.6 Feature filtering and selection

Before model fitting, a leak-free pre-filter removed degenerate and redundant columns using statistics computed on the training partition only and applied identically to the held-out set: near-constant columns below a variance threshold of 1×10^{-8} were dropped, binary columns whose positive prevalence fell outside 0.005–0.995 were removed, and exact-duplicate columns were pruned.

Selection was then performed train-only with a cross-validated elastic net. The `ElasticNetCV` selector was fit in an isolated subprocess with a wall-clock timeout over a configurable L1-ratio grid and log-spaced alpha grid, using folds matched to the active split strategy. Features were retained when the absolute coefficient exceeded a threshold, with at least one feature always kept and the retained set capped at the larger of one feature or 10% of the training-row count. To bound runtime, the selector used a dataset-size runtime model: when predicted elastic-net runtime exceeded a configurable threshold (default 7200s), or when the elastic net timed out or failed, selection fell back to `RandomForest` importance ranking. Selected features, coefficients and diagnostics were written per dataset.

2.7 Model library

Conventional machine learning.

For regression: elastic-net regression with internal cross-validated alpha/L1 search, support vector regression ($C = 10$, $\epsilon = 0.1$, RBF kernel), random forests (400 trees),

extremely randomized trees (500 trees), histogram gradient boosting (learning rate 0.05, up to 500 iterations, maximum depth 8), a soft-voting KNN+SVR regressor with adaptively chosen neighbour count, AdaBoost (500 estimators, learning rate 0.05), and a tabular multilayer perceptron (hidden layers 512 and 256, ReLU, Adam, L2 1×10^{-4} , up to 300 iterations), all via scikit-learn [38]. Gradient-boosting libraries were added when installed: XGBoost (400 trees, depth 6, learning rate 0.05, subsample and column-sample 0.9) [39], LightGBM (500 trees, learning rate 0.05, 63 leaves) [40] and CatBoost (400 iterations, depth 6, learning rate 0.05) [41]. Numeric pipelines were preceded by median imputation and standardization where appropriate. The classification table substituted the analogous estimators (logistic regression, SVC with probability estimates, random forest, extra trees, histogram gradient boosting, soft-voting KNN+SVC, AdaBoost, tabular MLP and the gradient-boosting classifiers) with classification losses and metrics.

Specialized tabular and graph models.

A compact one-dimensional convolutional regressor treats each standardized feature vector as a 1D signal and applies two same-padded ReLU convolutional blocks (64 filters, kernel size 5), global max pooling, a 128-unit dense layer with dropout and a linear output, trained with Adam and early stopping; it is intentionally small enough to run on CPU. The MapLight + GNN workflow fits a CatBoost model on the union of MapLight classic features and pretrained graph isomorphism network (GIN) fingerprints where the supporting DGL/PyTorch stack is available; a strict leaderboard-parity variant of the MapLight CatBoost model uses mean-absolute-error optimization, target scaling and five-seed averaging to reproduce the published protocol [14]. Graph neural networks were provided through Chemprop v2 [42, 43] with configured directed message-passing (D-MPNN), CMPNN-style and AttentiveFP-style architectures; optional variants augment the graph encoder with train-only selected tabular descriptors or an RDKit-2D featurizer. Uni-Mol V1 and V2 [10] were included as 3D pretrained baselines, running automatically when a GPU was detected. The configuration that produced the deposited results was 20 epochs, learning rate 1×10^{-4} , batch size 32, early stopping patience 5, a random internal validation split, a 64-atom cap and the 84m checkpoint, with automatic mixed precision enabled under an otherwise fp32 precision mode. These values are recorded in `run_config.json` and in the per-model `unimol.*` columns of every `metrics.csv`. The TabPFN tabular foundation model [44] was available for regression and classification, gated by a 1000-row training guardrail consistent with its design constraints.

Genetic-algorithm tuning (available but not evaluated).

An optional stage tunes a small set of estimators (elastic net and CatBoost for regression; elastic-net-penalized logistic regression and CatBoost for classification). It is disabled by default unless explicitly requested, or unless an `auto` mode finds prior evidence that a tuned family is worth rerunning. **It was disabled throughout the benchmark reported here** (recorded resolution: mode `disabled`, reason `empty_ga_models`), so no GA-tuned model appears in any result. We therefore draw no conclusion about what evolutionary tuning would or would not add: the capability

exists in the software and is simply untested in this study. Every result below comes from the fixed model library.

2.8 Combinatorial fusion and ensembling

After base models produced aligned train and test prediction vectors, QSARena optionally fused them by combinatorial fusion analysis (CFA) [45] in both score and rank spaces. To control combinatorial growth, inputs were first reduced to the best model per workflow and the subset search bounded by a budget guardrail. For each candidate subset the algorithm computed a performance strength as the inverse of the base model’s training error and a diversity strength from the mean pairwise distance between normalized, sorted score profiles, then evaluated three score-space weightings (equal, performance-weighted, diversity-weighted) plus the corresponding rank-space variants. Rank-space combinations were linearly calibrated back to the target scale and given a small metric discount when subset diversity exceeded a threshold, so that diverse rank fusions were preferred only when genuinely complementary. Candidates were ranked by the adjusted training metric and the best fused predictor returned with its selected models, weights and a candidate-diagnostics table. For regression, fusion minimized mean absolute error.

Three further ensemble strategies were available over the same prediction pool: out-of-fold stacking with a RidgeCV meta-model (logistic for classification), an inverse-training-RMSE weighted average (weighting by the primary classification metric for classification), and a simple average. Ensemble construction supported optional member filtering, including removal of highly correlated members and exclusion of members with negative held-out R^2 . We return to the implications of that last option in Section 3.3 and in the Limitations, because it consults held-out data.

2.9 Evaluation metrics

Regression performance was summarized by RMSE, mean absolute error, R^2 and Spearman rank correlation, computed separately on training and held-out sets; classification by AUROC, AUPRC, balanced accuracy and the Matthews correlation coefficient. Each dataset carries a primary metric: for cross-dataset ranking we used RMSE for regression and, for classification, the dataset’s designated primary metric, following the TDC leaderboard’s per-dataset assignments rather than a blanket rule. AUPRC is the primary metric on exactly five datasets — `tdc_cyp2c9_veith`, `tdc_cyp2d6_veith`, `tdc_cyp3a4_veith`, `tdc_cyp2c9_substrate_carbonmangels` and `tdc_cyp2d6_substrate_carbonmangels` — and AUROC is the primary metric on every other classification dataset, including `tdc_cyp3a4_substrate_carbonmangels`, which is a CYP substrate task but is scored by AUROC on the leaderboard. Four binary datasets (`tdc_cyp1a2_veith`, `tdc_cyp2c19_veith`, `tdc_herg_karim` and `tdc_pampa_ncats`) are mislabelled as regression tasks in our dataset catalog; the runner records a regression primary metric for them, but the analysis infers classification from strictly binary targets and uses the classification metrics throughout. The per-dataset assignment actually used is recorded in the `primary_metric` column of every `metrics.csv` and is listed in Table S1. For comparisons against published leaderboards we

re-selected the best model per dataset under the *leaderboard’s* metric, which is MAE or Spearman rather than RMSE for several TDC regression tasks; the best model under RMSE and under the leaderboard metric therefore sometimes differ, and both are reported (Additional file 1, Table S7 and Additional file 1, Table S1).

Because absolute metric differences are not comparable across datasets with different target units — an RMSE gap of 1.0 is negligible for hepatocyte clearance and fatal for log-solubility — all cross-dataset summaries of how close a model came to the per-dataset best use a **relative** gap, $|\text{score} - \text{best}|/|\text{best}|$.

2.10 Leaderboard comparison

Where curated leaderboard references existed, the best QSARena model was compared against published values using a normalized metric-matching procedure. Comparisons were made only when the leaderboard metric matched the dataset’s primary metric and a numeric reference value was available; the framework then computed the signed gap to the top-1 reference, the gap to the top-10 cutoff, and an estimated rank relative to the published entries. Reference rows were aggregated from TDC, MoleculeNet and Polaris leaderboards together with a manually curated set of TDC ADMET reference values and, for ESOL and Lipophilicity, current literature values in place of the 2017-era MoleculeNet baselines, which are no longer representative of the state of the art.

Two limits of this procedure should be kept in view. First, “estimated rank” reflects the references available for a dataset, not a live leaderboard submission; a dataset with few references yields an optimistic-looking rank. Second, ranks are only leaderboard-equivalent where the split protocol matches, which is true for the 22 TDC `admet_group` datasets and the 5 Polaris datasets but not for the 10 comparable datasets evaluated on locally generated splits. We report results stratified by this distinction throughout.

2.11 Model selection protocols

We report every headline result under two selection protocols.

Test-selected.

The best model per dataset is the one with the best held-out primary metric. This is the protocol used by AutoML leaderboard submissions generally, and it is what Sections 3.2–3.4 report as the primary analysis for comparability with published entries.

Cross-validation-selected.

The model is chosen per dataset solely by its cross-validated primary metric on the training partition, and its held-out score is then read off without further choice. This estimates what a practitioner obtains when the test set is genuinely untouched. It is available for the models that emit cross-validated metrics — the conventional estimators, TabPFN and the ChemML MLP — but not for Chemprop, Uni-Mol, Map-Light + GNN or the fusion methods, so it is a conservative lower bound on what a fully CV-driven QSARena would achieve.

2.12 Reproducibility, caching and computing environment

The runner is designed for long, resumable runs. Completed datasets and compatible intermediates are reused on resume, and each run records a configuration signature, per-dataset run-status manifests, split-signature hashes and per-stage runtime diagnostics. Two cost profiles are provided: a default cost-optimized profile that disables historically low-value expensive variants, and a full profile restoring the broader model set. Each dataset directory retains metrics, predictions, selected features, a feature-deduplication report, the CFA candidate table and ensemble weights, supporting independent verification of every reported result.

The workflow targets Python 3.11 and runs on Windows, macOS, Linux or Google Colab. The core workflow runs CPU-only; GPU availability primarily affects runtime and whether optional backends such as local TabPFN and Uni-Mol are practical. Pinned conda and pip/uv specifications are distributed with the repository, alongside an Apptainer/Singularity definition, Slurm submission scripts for NSF ACCESS HPC clusters, and an OpenStack orchestration path for Jetstream2 [46, 47] that routes GPU-dependent workflows to A100-backed instances and the remaining workflows to CPU instances under a tracked service-unit budget.

The benchmark reported here was executed on NSF ACCESS Jetstream2 [46, 47] under allocation CIS261142, on a `g3.large` instance providing one NVIDIA A100-SXM4-40GB GPU (39.5 GB device memory, 108 streaming multiprocessors, compute capability 8.0) and 32 CPU cores, running Linux with all timestamps in UTC. The device inventory, CPU count, parallelism settings and `fp32` precision mode are recorded in the run’s `run_timing.json` and `run_config.json` and are reproduced in the deposited artifacts. Section 3.10 additionally reports the same benchmark executed on a consumer laptop GPU, as a direct test of whether the results depend on datacentre hardware.

All figures, tables and numerical claims in this paper are regenerated from committed benchmark artifacts by a single command, `python portable_colab_qsar_bundle/render_manuscript_assets.py`, which re-executes the analysis notebook and writes the figures, tables and a `manuscript_numbers.json` containing every headline number quoted in the text. A companion script, `verify_manuscript_numbers.py`, asserts that the manuscript text still matches those artifacts.

3 Results and discussion

3.1 Benchmark coverage

We executed QSARena across the full benchmark suite under a single fixed configuration (`full` profile, random seed 13, Chemprop seed 42) on an NVIDIA A100-SXM4-40GB GPU with 32 CPU cores, provided by NSF ACCESS Jetstream2. Of 45 datasets attempted, 43 completed; `tdc_herg_central`, the largest task attempted, was abandoned after running more than 24 hours without completing, and `polaris_adme_fang_hppb_1` was interrupted when the run ended but had already produced a full model table and is retained. The 44 analysed datasets comprise 22 regression and 22 classification tasks drawn from five collections: TDC (32), Polaris ADME (5),

MoleculeNet (3), ChemML (2) and PODUAM (2). They span 280 to 13,445 molecules (median 1,605; 156,052 in total) and four split protocols (27 predefined, 12 scaffold, 4 target-quartile, 1 random). Twenty-eight models produced at least one valid result, yielding 837 valid model-dataset evaluations. The dataset catalog, with each dataset’s estimated leaderboard rank and the best published value it is measured against, is given in Additional file 1, Table S6.

3.2 No single model family dominates

The central finding is that no model family won across the benchmark (Fig. 3, Table 2). Stacking and averaging ensembles produced the most outright wins (16 of 44 datasets: 9 classification, 7 regression), followed by the 3D pretrained Uni-Mol models (11: 5 classification, 6 regression), conventional machine learning (8: 6 classification, 2 regression), the MapLight + GNN descriptor-graph hybrid (4, all regression), deep tabular networks and Chemprop v2 (2 each) and combinatorial fusion (1). The largest single share is 36% of datasets. We flag immediately that the ensemble row is not a like-for-like competitor: ensembles are built *from* the other families’ predictions, and under the configuration used here their member filtering consults held-out R^2 , so their lead should be read as “the pipeline’s output is reliably near-best” rather than as evidence that stacking is intrinsically superior to its members (Sections 3.3 and 3.5).

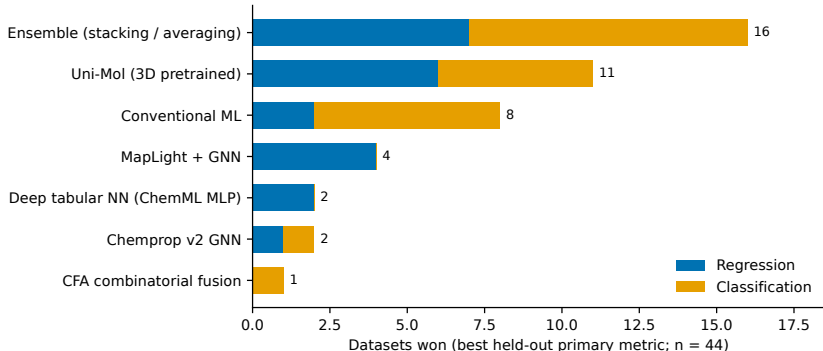


Fig. 3 Datasets won by each model family, split by task kind. A win is the best held-out primary metric on that dataset among all 28 models. *Single split and single seed: individual placements are provisional and no variance is estimated (see Limitations).*

The distribution of winners is more even than in our earlier consumer-GPU run (Section 3.10), and one difference is worth stating explicitly because it reverses a conclusion we drew from that run: with a longer training schedule (20 epochs, batch size 32, mixed precision) and the addition of Uni-Mol V2, the 3D pretrained family won five classification datasets (`tdc_cyp2d6_substrate_carbonmangels`, `tdc_dili`, `tdc_pampa_ncats`, `tdc_pgp_broccatelli`, `tdc_skin_reaction`) as well as six regression datasets. Conformer-based pretraining is therefore not a regression-only tool, and any claim that it categorically fails on classification is an artifact of undertraining it. The regression datasets it won remain chemically coherent — permeability, clearance,

lipophilicity and aqueous solubility — properties for which conformational and shape information is mechanistically plausible as signal that 2D descriptors capture only indirectly.

Per-dataset winners, including the corresponding cross-validation-selected model, are listed in Additional file 1 (Table S1), and the full dataset $\times$ family landscape is shown in Fig. 7. These are single-split, single-seed outcomes: an unknown fraction of individual winners would change under a different seed, so we read Fig. 3 as a distribution of capability across families rather than as a precise ranking, and we base the claim below on its breadth rather than on any one dataset’s result (see Limitations).

3.3 Consistency versus peak performance

Win counts reward only the single best model per dataset and understate the reliability of models that are consistently near-best without winning. Table 2 therefore reports, for each family, the median relative gap to the per-dataset best, the fraction of datasets on which the family’s best member landed within 5% of the best result, and the median rank that member achieved.

Table 2 Model-family coverage and consistency. Gaps are relative to the per-dataset best primary metric. “Within 5% of best” counts datasets where the family’s best member fell within 5% of the dataset winner. Families evaluated on fewer than 44 datasets were limited by task applicability, size guardrails or backend failures (Additional file 1, Table S4); their percentages are computed over the datasets on which they ran. All values derive from a single split and seed, so adjacent rows are not separated (see Limitations).

Model family	Models	Datasets with valid results	Wins (regression)	Wins (classification)	Median gap to best, regression (%)	Median gap to best, classification (%)	Within 5% of best (% of datasets)	Median rank of family-best model
Ensemble (stacking / averaging)	2	44	7	9	1.6	0.3	75.0	2.0
Uni-Mol (3D pretrained)	2	44	6	5	3.8	1.7	70.5	4.0
Conventional ML	15	44	2	6	2.8	1.5	65.9	3.0
MapLight + GNN	1	44	4	0	7.5	11.1	27.3	8.5
Chemprop v2 GNN	5	6	1	1	10.1	1.0	83.3	2.0
Deep tabular NN (ChemML MLP)	2	44	2	0	18.2	7.5	25.0	12.5
CFA combinatorial fusion	1	44	0	1	6.2	1.8	63.6	4.5

Ensembles were within 5% of the best on 75% of datasets with a median rank of 2; the 3D pretrained family followed closely at 70% and median rank 4, and conventional machine learning at 66% and median rank 3. The gap between these three is not resolvable at single-seed resolution, and we do not claim an ordering among them. Further back, MapLight + GNN (27% within 5%, median rank 8.5) and the deep tabular networks (25%, median rank 12.5) were markedly less consistent despite MapLight + GNN winning four datasets outright — a bimodal profile in which a model is either the best available or well down the table.

Chemprop v2 shows the highest within-5% rate in the table (83%) and a median rank of 2, but on only 6 of 44 datasets, because four of its five configured variants failed on most tasks (Section 3.8). That number says Chemprop is strong where it ran, not that it is the most consistent family; it is the one row in Table 2 that should not be compared with the others.

Three further cautions apply. Families with more models have more chances to produce a near-best member, so the 15-model conventional family is flattered relative to single-model families; the ensemble row is not a like-for-like competitor, because ensembles are built *from* the other families’ predictions and, under the configuration used here, their member filtering consults held-out R^2 ; and the ensemble family’s apparent consistency should therefore be read as “the pipeline’s output is reliably near-best”, not as evidence that stacking is intrinsically superior to its members. Section 3.5 quantifies how much the fusion layer actually adds.

3.4 Leaderboard competitiveness, and the cost of honest model selection

Across the leaderboard-comparison layer, 37 datasets could be compared against 430 published reference values. The best QSARena model per dataset placed within the estimated top ten on 35 of 37 datasets, with a median estimated rank of 3 and five estimated first places (`tdc_bioavailability_ma`, `tdc_carcinogens_lagunin`, `tdc_clearance_microsome_az`, `tdc_cyp2c9_substrate_carbon-mangels`, `tdc_toxcast`). The two datasets below the top ten were `tdc_skin_reaction` and `tdc_tox21`, both evaluated on locally generated scaffold splits that are harder than the references they are scored against.

Comparability varies by dataset, and the aggregate obscures it. Restricted to the 22 TDC datasets evaluated on official `admet_group` splits — the only subset directly comparable to the public TDC leaderboard — the test-selected model placed in the top ten on **all 22**, with a median rank of 3 and three estimated first places. On the 5 Polaris datasets it placed in the top ten on all 5, median rank 3. The remaining 10 comparable datasets used locally generated splits and are *not* leaderboard-equivalent; both sub-top-ten results and two of the five first places fall in this group, and those should be read as “competitive with published values under a comparable but not identical protocol”, not as leaderboard claims. Table 3 gives the breakdown by comparison class; per-dataset detail, including both selection protocols and the reference counts behind each rank, is given in Additional file 1, Table S7.

That headline, however, is produced by choosing the best of 28 models using held-out scores. Under the stricter protocol in which the model is chosen by cross-validation

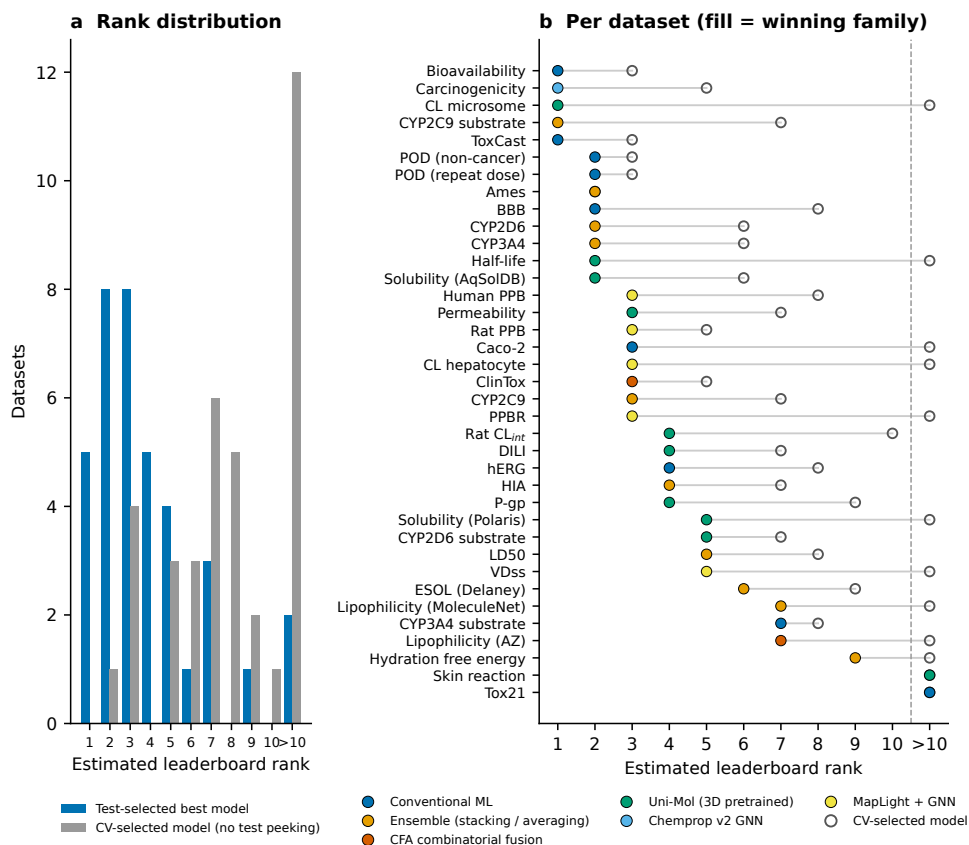


Fig. 4 Estimated leaderboard placement across 37 comparable datasets. (a) Rank distribution for the test-selected best model and for the model selected by cross-validation alone. (b) Per-dataset ranks; filled markers are the test-selected model coloured by model family, open markers the cross-validation-selected model. The dashed line marks the top-ten boundary. *Single split and single seed: individual placements are provisional and no variance is estimated (see Limitations).*

Table 3 Rank among curated published reference values, by comparison class. Only the 27 datasets with official predefined splits — 22 TDC ADMET Benchmark Group and 5 Polaris ADME — are directly comparable to a public leaderboard; the remaining 10 are scored against published values under a comparable but not identical protocol and are reported separately rather than pooled. Columns: “n” is the number of datasets in the class; “test” selects the best model per dataset on held-out data; “CV” selects by cross-validation only. Per-dataset detail is in Additional file 1, Table S7. Single split and single seed, so individual placements are provisional.

Comparison class	n	Top-10 test	Median rank test	Top-10 CV	Median rank CV
TDC ADMET Group (official)	22	22	3.0	15	8.0
Polaris ADME (official)	5	5	3.0	4	8.0
Local splits (not leaderboard-equivalent)	10	8	4.5	6	7.0
All comparable datasets	37	35	3.0	25	8.0

alone (Section 2.11), top-ten placement falls from 35 to 25 of 37 datasets, first places from five to none, and the median estimated rank from 3 to 8 (Fig. 4). On the official TDC subset the fall is from 22 of 22 to 15 of 22. Across all 44 datasets, the cross-validation-selected model was never the overall winner and sat a median of 16.1% above the per-dataset best.

That ten-dataset gap has two causes, and separating them changes its interpretation. Cross-validation selection is restricted to models that emit cross-validated metrics, which in this run excludes Chemprop, the Uni-Mol models, MapLight + GNN and the fusion methods. The comparison above therefore confounds the cost of not peeking at the test set with the cost of losing those candidate families. To separate them we added a third protocol that holds the candidate set fixed at the cross-validation-eligible models and selects among them on the test metric:

Selection	Candidate set	Top ten	First places	Median rank
Test metric	Full 28-model library	35/37	5	3
Test metric	CV-eligible models only	28/37	3	6
Cross-validation	CV-eligible models only	25/37	0	8

Read down the table, **7 of the 10 lost placements come from narrowing the candidate set and only 3 from honest selection**; the median rank moves from 3 to 6 on library breadth and from 6 to 8 on selection protocol. Both effects are real and neither is small, but they are different claims. The breadth of the model library is worth about seven leaderboard placements, which is the strongest available argument for running many model families rather than one well-tuned architecture. The cost of honest model selection is about three placements and, more strikingly, every one of the five first places: **no protocol that refuses test-set information produced a single estimated rank of 1**. We previously reported the whole ten-dataset gap as the cost of honest selection; that attribution was wrong, and the corrected decomposition is the more useful result.

Both protocols rest on a single split and seed, so the individual ranks in Fig. 4 carry unquantified seed variance and the aggregate counts are the more trustworthy quantity (see Limitations). That caveat does not soften the direction of the effect: the shift is large and one-sided, and it would take implausibly favourable seed variance to erase a ten-dataset gap. This gap is, in our view, the most practically important number in this paper. It is not a defect specific to QSARena: any leaderboard entry that selects among candidate models or configurations using test-set feedback carries the same inflation, and the reproducibility audit of the TDC leaderboard [20] suggests that such selection is rarely documented. We report both protocols rather than only the favourable one, and we take the cross-validation-selected result — top ten on roughly two-thirds of comparable datasets — as the honest estimate of what a practitioner should expect. The cross-validation protocol is also a conservative bound here, since Chemprop, the Uni-Mol models, MapLight + GNN and the fusion methods do not emit cross-validated metrics in this run and were therefore ineligible for cross-validation-based selection; extending CV metrics to those backends is the single most valuable change we can make to the runner.

The reference set is itself imperfect, and this biases the comparison. The audit we cite above [20] examined the top-ranked TDC ADMET entries and found that only three — CaliciBoost, MapLight and MapLight + GNN — passed all reproducibility and leakage checks, reporting direct or indirect data leakage in several others and noting that deliberate or accidental test-set tuning can elevate mediocre models to top-tier leaderboard standing. Our estimated ranks are computed against exactly those published values. The consequence is a bias against QSARena of unknown size: a cross-validation-selected model is being ranked against a reference population that includes entries inflated by the very selection effect we measure in this section. This cuts in a specific direction — it makes our test-selected numbers *less* impressive than they appear relative to honest competitors, and our cross-validation-selected numbers *more* conservative than a like-for-like comparison would give. We therefore treat the gap between the two protocols, which is internal to our own run and free of this contamination, as the more trustworthy quantity, and we recommend that future leaderboard-relative claims be scored against an audit-verified subset once one is maintained.

Two datasets warrant specific comment. For ESOL and Lipophilicity we deliberately replaced the public MoleculeNet leaderboard — whose only entries are a 2020-dated GCN and random forest — with current literature values, because scoring against the older baselines produced spurious first places in an earlier analysis of this work. Against contemporary references QSARena reaches RMSE 0.617 on ESOL and 0.589 on Lipophilicity. These are competitive results and we make no state-of-the-art claim for either.

3.5 What the pipeline’s components contribute

Because fusion and ensembling add computational cost and interpretive complexity, we asked what each layer of the pipeline actually buys (Table 4).

Table 4 Ensemble and fusion value-add against the best single base model available on the same dataset, under each dataset’s primary metric.

Fusion method	Task	Datasets	Overall wins	Top-3	Beats best base	Loses to best base	Median rank	Median rel. change vs best base
OOF stacking	classification	22	6	13	8	14	3.000	-0.003
Inverse-RMSE	classification	22	3	10	7	15	4.000	-0.008
weighted average								
CFA fusion	classification	22	1	10	5	17	4.000	-0.009
Inverse-RMSE	regression	22	6	16	7	15	2.500	-0.016
weighted average								
OOF stacking	regression	22	1	6	4	18	6.000	-0.050
CFA fusion	regression	22	0	5	1	21	6.000	-0.051

Taking the best fusion method per dataset, some fusion beat the best single model on 10 of 22 classification datasets, with a median relative improvement of 0.8% where it won, and on 7 of 22 regression datasets, with a median improvement of 2.4% where it won. Across all datasets the typical fusion result was slightly worse than the best single model (median -0.1% for classification, -1.4% for regression), because fusion also loses on the datasets where it does not win. The directional conclusion is that fusion is worth its cost roughly half the time on classification and a third of the time on regression, and that a practitioner who can afford to run it should, but should not expect it to win by default. These margins are the smallest quantities we report — median relative changes below 3% on a single split and seed — and are correspondingly the most fragile.

A staged ablation over the same artifacts (Additional file 1, Table S2) tells a consistent story: starting from conventional machine learning alone, adding MapLight classic features improved the achievable result on 10 of 44 datasets, adding the neural and pretrained backends improved 26, adding CFA improved 6, and adding the ensemble layer improved 16. The deep and pretrained backends are therefore the single largest source of incremental accuracy — worth stating plainly, since it cuts against a purely “conventional models are enough” reading of Fig. 3 — while the fusion layers contribute more modestly. What Fig. 3 and Table 2 add is that this incremental accuracy is not free (Section 3.7) and does not make any pretrained family a reliable default.

3.6 Feature representations

We quantified which representation families the leak-free elastic-net selector actually retained, relative to a uniform-selection baseline (Fig. 5; Additional file 1, Table S3). Of 12,137 selected features across all datasets, the three components of the MapLight classic composite accounted for 46.2%, against 22.9% of the available feature pool — a clear enrichment for the composite as a whole.

On a per-feature basis the ordering is more informative. The compact RDKit 2D descriptor panel was by far the most enriched family ($5.16\times$ uniform), followed by ErG ($2.86\times$) and Avalon ($2.54\times$); MACCS keys were marginally enriched ($1.18\times$); and the large hashed circular and path fingerprints were all *de*-enriched, including FCFP6 ($0.73\times$). In other words, a few hundred interpretable physicochemical descriptors earn selection far above their numerical weight, while thousands of hashed fingerprint bits earn it below theirs. Avalon’s large absolute share (29.7% of selected features) reflects its size as much as its per-feature value.

This nuance matters for how the MapLight result is usually described. The composite’s value comes substantially from its two compact, chemically interpretable components — the ErG pharmacophore fingerprint and the RDKit descriptor panel — rather than from its hashed Morgan block. For practitioners with limited compute, the practical implication is that the RDKit descriptor panel plus one compact pharmacophore representation captures most of the selectable signal, a conclusion consistent with representation-focused benchmarks reporting that descriptor sets outperform fingerprints as standalone representations [17]. This is consistent with a systematic study of molecular property prediction reporting that fixed representations lead on most

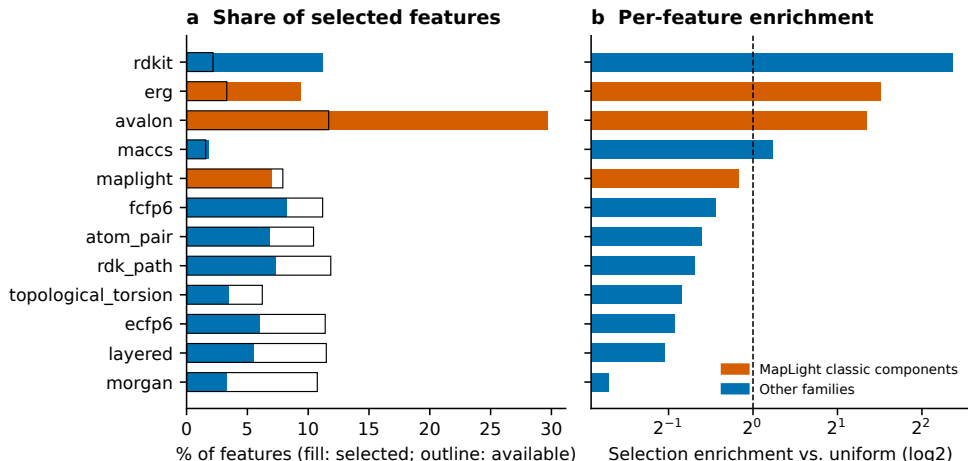


Fig. 5 Feature-family representation among selected features. (a) Share of selected features (filled) against share of available features (outlined). (b) Per-feature selection enrichment relative to a uniform baseline, log₂ scale. Components of the MapLight classic composite are highlighted.

datasets, that SVMs remain competitive with neural networks, and that dataset size is a key bottleneck for representation-learning models [48].

3.7 Computational cost

A core motivation for QSARena is that competitive accuracy should not require GPU-scale compute. The complete benchmark consumed 111.8 hours of recorded wall-clock time across 44 datasets (median 1.89 h per dataset; maximum 11.2 h for `tdc_herg-karim`, 13,445 molecules) on the A100 node. We report per-model cost as each model’s own incremental wall-clock time, computed as the difference between consecutive entries of the runner’s cumulative per-session clock; fusion methods are additionally charged the summed cost of the base-model pool they consume, since a fusion result cannot be obtained without it.

Per-family median cost spans more than three orders of magnitude (Table 5): 0.3 s for CFA and 0.6 s for the ensemble meta-models (excluding their base pool), 6.8 s for conventional machine learning, 39.8 s for the deep tabular networks, 137 s for MapLight + GNN, 269 s for Chemprop v2 and 374 s for the Uni-Mol models. The 3D pretrained family therefore costs roughly 55 times the median conventional model per dataset, in exchange for 11 wins out of 44 and a median rank of 4. That is a far better trade than the same comparison made under the shorter training schedule of our consumer-GPU run, and it is the clearest argument in this paper for spending GPU time — but it remains a trade to make deliberately, not a default.

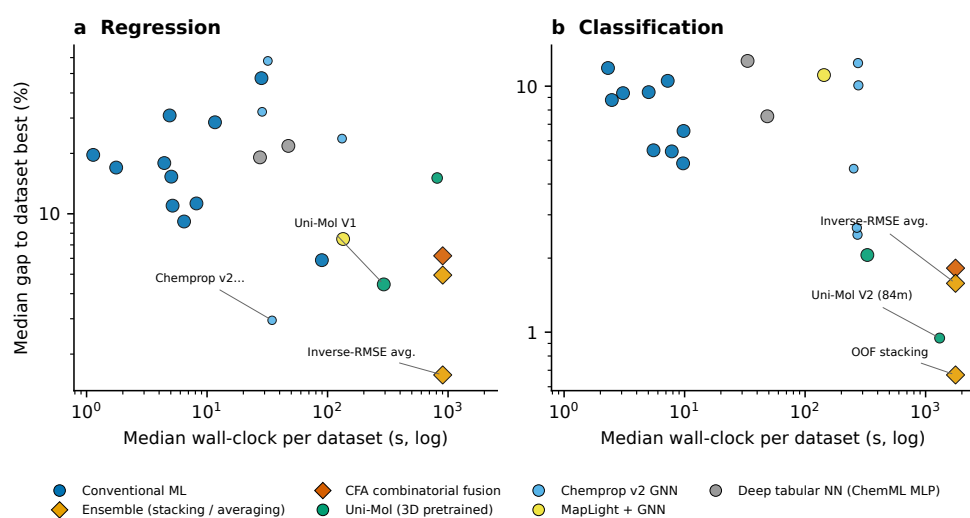


Fig. 6 Median wall-clock time per dataset against median relative gap to the per-dataset best, by model, separately for regression (a) and classification (b). Both axes are logarithmic; diamonds mark fusion methods, which are charged the summed cost of the base-model pool they consume in addition to their own, since a fusion result cannot be obtained without it.

Table 5 Per-family computational cost and model size in this benchmark, with published comparators. Wall-clock times are per model per dataset on the run hardware.

Model family	Model-dataset fits timed	Median own wall-clock (s)	IQR own wall-clock (s)	Median cost incl. base pool (s)	Median trainable parameters	Notes
CFA combinatorial fusion	44.0	0.3	0--0	1,378		fusion over fitted base-model predictions
Ensemble (stacking / averaging)	87.0	0.6	0--1	1,378		fusion over fitted base-model predictions
Conventional ML	440.0	6.8	3--20		157,953	measured in this run
Deep tabular NN (ChemML MLP)	88.0	39.8	27--94		90,369	measured in this run
MapLight + GNN	44.0	137.4	126--160			measured in this run
Chemprop v2 GNN	25.0	268.6	229--274		325,552	measured in this run
Uni-Mol (3D pretrained)	61.0	373.5	170--1285		47,331,652	measured in this run
MolGPS (published)						~3B parameters; GPU pretraining/inference reported in literature
MolE (published)						~100M parameters; GPU pretraining reported in literature
ADMET-AI (published)						Chemprop-RDKit; exact parameter count not recorded here; GPU-capable Chemprop-RDKit deployment

Recorded model sizes underline the accessibility argument. The models that won most of our datasets — gradient-boosted trees and the linear meta-models over them — carry no pretraining corpus at all, against published comparators MolE (~ 100 M parameters, pretrained on ~ 842 M molecules) [7] and MolGPS (~ 3 B parameters) [8].

Feature selection scaled sub-linearly with dataset size in this run: a log-log fit of selector time against dataset size gives a slope of 0.77 (Pearson $r = 0.58$ across 44 datasets), with a median of 295 s and a maximum of 1,003 s on `tdc.herg.karim`. This run disabled the dataset-size RandomForest fallback (Section 2.6), so every dataset used the full elastic-net selector; the flatter scaling relative to our consumer-GPU run reflects 32-way parallelism rather than a change in the algorithm.

3.8 Model coverage and backend failures

Not every model ran on every dataset, and we report the gaps rather than silently analysing only successes (Additional file 1, Table S4). Four sources account for nearly all of them. Task applicability: regression-only and classification-only estimators are valid on at most 22 datasets each. Configuration: TabPFN was disabled in this run, so the tabular foundation model is absent from the comparison entirely. Guardrails and capacity: Uni-Mol V2 produced valid results on 17 of 44 datasets at the 84 M-parameter size, and its 164 M and 310 M variants produced none, so the V2 results here are a partial view of that architecture. Backend failures: four of the five configured Chemprop v2 variants failed on most datasets, leaving all Chemprop variants valid on only 6 of 44.

The Chemprop coverage is the most consequential gap for interpretation. Chemprop’s two wins and its high within-5% rate (Table 2) are measured over those 6 datasets; a working Chemprop path across the full suite could change its standing substantially in either direction. We flag this as the largest known threat to the completeness of Fig. 3 and as the first thing to fix before any follow-up benchmark, together with restoring TabPFN and diagnosing the larger Uni-Mol V2 variants.

3.9 Reproducibility

Every reported result is backed by a complete artifact trail. Each analysed dataset retains metrics, selected-feature records, selector coefficients, split-signature hashes, per-stage runtimes, CFA candidate tables and ensemble weights; the run recorded a configuration signature, fixed seeds (random seed 13, Chemprop seed 42), a full benchmark profile, fp32 precision and a GPU inventory identifying the A100 device; and a SHA-256 manifest covers repository-level and per-dataset artifacts. The benchmark artifacts analysed here were committed in repository revision `bbfb188`. All figures, tables and quoted numbers in this paper regenerate from those artifacts with one command (Section 2.12), which also writes a machine-readable record of every headline number, and a companion script asserts that the manuscript text still matches it.

One limitation of the reproducibility claim should be stated: per-molecule prediction files are excluded from the repository for size reasons, so prediction-level diagnostics (for example inter-model prediction diversity) cannot be reproduced from

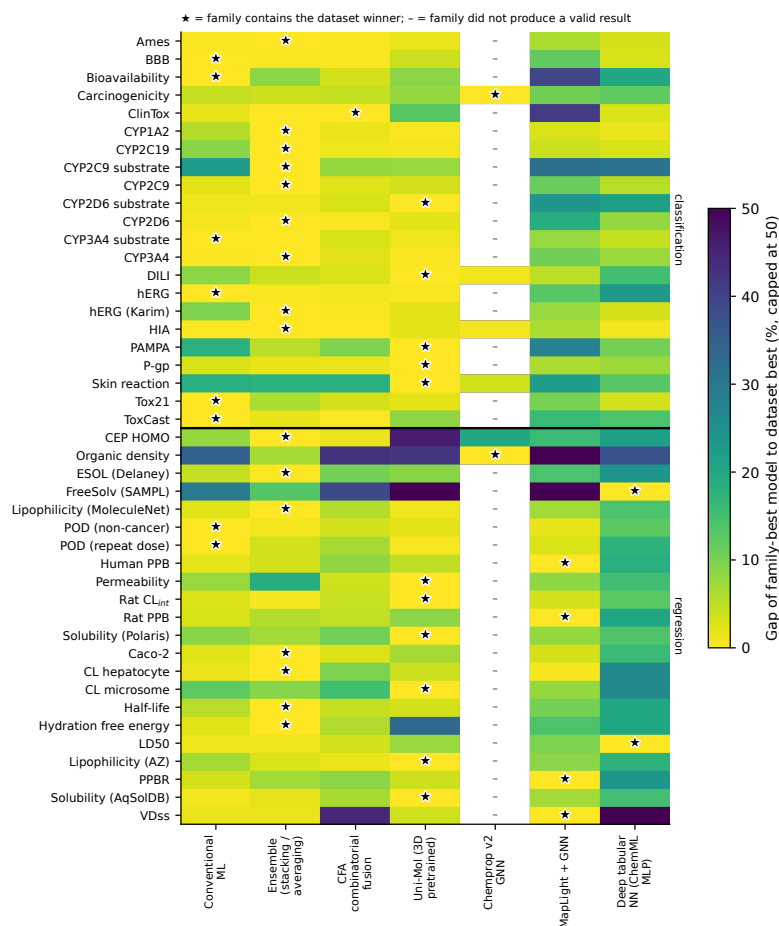


Fig. 7 Relative gap of each family’s best model to the per-dataset best (%), capped at 50, for every dataset. Stars mark the family containing the dataset winner; dashes mark families that produced no valid result for that dataset. Classification datasets are shown above the rule, regression below.

the public artifacts alone. These are included in the archived release described under Availability of data and materials.

3.10 Hardware sensitivity: datacentre GPU versus consumer laptop GPU

Because accessibility is a central claim of this work, we repeated the analysis on an earlier execution of the same benchmark performed on a consumer laptop GPU (NVIDIA GeForce RTX 4060 Laptop GPU, Windows, `cost_optimized` profile) and compared the two runs dataset by dataset under identical analysis code (Additional file 1, Table S5).

Thirty-seven of the 44 datasets used byte-identical held-out partitions across the two runs, verified by split-signature hash; the other seven were re-split from random

or target-quartile to scaffold splits in the A100 run and are therefore not directly comparable. On those 37 directly comparable datasets, the best available model changed the primary metric by a median of -0.19% — the consumer-GPU run was better on 21 datasets and the A100 run on 15. In other words, the two hardware configurations are indistinguishable in accuracy at this resolution.

What the A100 bought was not accuracy but capacity and throughput: a **full** rather than cost-optimized model profile, a longer Chemprop schedule (40 epochs, 3-model ensembles versus 15 epochs and single models), a longer Uni-Mol schedule with mixed precision, the addition of Uni-Mol V2 and a TensorFlow deep tabular backend, 32-way parallel feature selection, and a complete run in 112 hours rather than 155. Those changes are what produced the shifts in *which family wins* reported in Section 3.2 — most notably the 3D pretrained family’s five classification wins, which did not occur under the shorter schedule.

The practical implication for readers is favourable to the accessibility argument. A practitioner with a laptop GPU can expect the same accuracy from this pipeline as one with an A100; what they lose is the ability to train the most expensive backends to convergence, and the wall-clock headroom to search as widely. The headline leaderboard placement in Section 3.4 does not depend on datacentre hardware.

3.11 Effort required of the user

Two properties of the benchmark design bear directly on what a practitioner should expect, and we report them here because they are results of this study rather than claims about the software.

First, **every number in this paper was produced under a single fixed configuration**. The same profile, seeds, feature families, selector and model library were applied to all 44 datasets, spanning five suites, two task types and a 48-fold range of dataset size. No dataset received bespoke feature engineering, model choice or hyperparameter tuning, and the optional genetic-algorithm search was disabled throughout (Section 2.7). The estimated top-ten placement on 35 of 37 leaderboard-comparable datasets was therefore obtained without per-dataset iteration. For a practitioner this is the operative number: it estimates what one run on a new dataset yields, not what an expert reaches after a tuning campaign. The honest counterpart remains the cross-validation-selected protocol of Section 3.4 — top ten on 25 of 37 — which additionally removes the benefit of picking the winner on held-out data.

Second, **obtaining that result requires no local installation or hardware**. The notebook entry point runs end-to-end in Google Colab on the free tier, where the user supplies a CSV of SMILES and targets and selects columns through form widgets; no code is written or edited, no environment is configured, and no local GPU or CPU is used. The command-line runner is the same workflow core for batch or HPC use, and Section 3.10 shows that accuracy does not depend on which of these paths is taken or on the class of hardware underneath.

We note the limits of this claim. The fixed configuration was chosen before the benchmark was run, but it was informed by our own earlier development on overlapping datasets, so it is not a blind default in the strictest sense. Datasets far outside the size and endpoint range sampled here may need attention that these 44 did not. And

the code-free path exposes a deliberately narrow set of choices; users wanting unusual splits, custom featurization or new backends will need the command-line interface and a local environment.

3.12 Comparison with existing automated QSAR platforms

Automating the QSAR model-building loop is not a new idea, and QSARena should be positioned against the two most directly comparable prior systems rather than only against single-model leaderboard entries.

QSAR Workbench [49] is the closest intellectual ancestor. Built on Pipeline Pilot, it systematically explores combinations of descriptor sets, learners and train/test splits, generating on the order of 600 models in under an hour, and it reports performance equivalent to published state-of-the-art approaches on its demonstration data. Its authors are explicit that the system has no built-in competitive model selection and that choosing among the generated models requires human judgement. That is precisely the step QSARena automates and then audits: the per-dataset winner is selected programmatically, and Section 3.4 quantifies how much of the resulting apparent performance is an artifact of making that selection on held-out data. QSAR Workbench is commercial, depends on a proprietary workflow engine, and was demonstrated on two datasets (209 and ~ 670 compounds); QSARena is MIT-licensed, depends only on open Python packages, and is evaluated here on 44 datasets spanning 280 to 13,445 molecules.

Schrödinger AutoQSAR [50] and its successor DeepAutoQSAR occupy the same application area as this work. AutoQSAR automates model building, validation and deployment, runs on Windows, macOS and Linux through either a command line or the Maestro GUI, and is distributed as closed-source software within the commercial Schrödinger Suite. Schrödinger has published a benchmark of DeepAutoQSAR against ChemProp and DeepPurpose on the ADMET subset of the Therapeutics Data Commons [51], reporting that DeepAutoQSAR ranked among the top performers on 20 of 22 cases and clearly outperformed the comparators on 9 of them.

A direct accuracy comparison against that study is not possible, and we do not attempt one. The white paper defines “top performer” and “clearly outperforming” qualitatively rather than as a numerical criterion, does not report per-dataset metric values, and does not release per-molecule predictions, so its results cannot be re-scored under our protocol or ours under theirs. What can be compared is summarized in Table 6, and it is less than it first appears. DeepAutoQSAR’s “top performer on 20 of 22” is its standing against two named comparators, ChemProp and DeepPurpose, not a position on the TDC leaderboard; QSARena’s 22 of 22 is a rank among curated published reference values. The two numbers are not commensurable and we do not treat them as such. What the two systems can be compared on is every axis that governs who can actually use them, and there they differ sharply.

Three differences deserve emphasis. On **cost and access**, the Schrödinger tools require a commercial licence and QSAR Workbench requires a Pipeline Pilot licence, whereas QSARena is free, MIT-licensed, installable with `pip`, and runnable without any installation at all in a hosted notebook (Section 3.11). On **transparency**, the comparators’ benchmark evidence is a vendor white paper and a demonstration study

respectively; QSARena’s is a deposited artifact trail from which every number in this paper regenerates, together with a script that fails if the text and the artifacts disagree. On **breadth**, both prior systems are single-suite; QSARena is evaluated across five.

We also note what the commercial tools plausibly do better. Neither the QSAR Workbench nor the Schrödinger products carry the coverage gaps we report in Section 3.8, both are professionally supported, and DeepAutoQSAR’s benchmark reflects sustained engineering investment in a single well-tuned engine rather than the broad-but-shallow sweep we perform. A reader whose priority is a supported production pipeline on one endpoint class may reasonably prefer them; a reader who needs auditability, cross-suite breadth, or zero cost should prefer this work.

Relationship to ADMET-AI, stated precisely. ADMET-AI is the reference open tool in this space, and the relationship is easy to misread, so we set it out exactly. ADMET-AI is *not* one of the 28 models evaluated here; it appears only as a cited comparator. What QSARena integrates is the package ADMET-AI is built on: our Chemprop v2 (D-MPNN + RDKit2D) variant concatenates a Chemprop graph representation with RDKit physicochemical descriptors, which is essentially ADMET-AI’s recipe, but it is a reimplementaion through Chemprop v2 rather than ADMET-AI’s own code, package or trained weights.

That path largely failed in this run. All five configured Chemprop variants, including the ADMET-AI-equivalent one, produced valid results on only 6 of 44 datasets (Section 3.8). **We therefore cannot claim either to reproduce or to beat ADMET-AI: parity is untested, not lost.** We ran no head-to-head comparison against it, and none should be inferred from the estimated ranks reported here. The same applies to the title’s claim that conventional and ensemble methods “match pretrained molecular models”: that claim is supported for the pretrained models we did run to convergence (Uni-Mol V1 and V2), and is *not* established for ADMET-AI specifically. Repairing the Chemprop backend so the D-MPNN + RDKit2D variant runs across all 44 datasets would turn it into a genuine like-for-like ADMET-AI proxy, and is the single most valuable change we can make to the comparison (Section 4).

Where QSARena does not lead, and how we know. Comparing against published ADMET systems is harder than the literature makes it appear, and getting it right changed our own conclusions, so we set out the procedure. Published rankings are almost always each paper’s rank *at its own publication date*. Our curated reference table inherits that convention: it contains 64 rows claiming first place across 28 datasets, and **caco2_wang** alone carries four different models recorded as rank 1. Such figures cannot be compared with one another, and none of MaxQsaring, ADMETboost, ADMET-AI, Auto-ADMET or DeepAutoQSAR appears in the TDC leaderboard snapshots we captured, so no head-to-head measurement against them exists.

We therefore re-scored every reference value in the honest direction for its metric, producing one consistent ranking per dataset. This is a materially different picture from the published claims. MaxQsaring, whose paper reports first place on 19 of 22 TDC tasks [16], holds 7 first places under joint re-scoring but is top-3 on 19 of 22 with a median rank of 2 — the strongest system in the reference set, and the fairer way to state its lead. ADMETboost’s reported 18 first places [13] reduce to **zero**: it was genuinely first in 2022 and has since been overtaken on all 22 datasets, which is a

caution about citing leaderboard positions as though they were durable. MapLight’s top-3 placement on 16 of 22 [14] is an author self-report we cannot re-score, and DeepAutoQSAR’s “top performer on 20 of 22” [51] is best-of-three against ChemProp and DeepPurpose in a vendor white paper rather than a leaderboard position at all — the two figures were never in competition. We also corrected three reference values that were on the wrong scale, all from a model reporting normalised targets (for example a PPBR MAE of 0.679 against a leaderboard best of 7.440).

Against that corrected reference set, QSARena’s best model per dataset reaches a median rank of 3 on the 22 official TDC tasks under test-based selection, and falls to a median rank of 8 under cross-validation-only selection (Section 3.4). MaxQsaring’s median rank of 2 is therefore better than ours under either protocol. We make no claim to accuracy leadership: the contribution is the framework, the finding about selection protocol, and the access.

The same landscape view explains why “rigorous benchmarking” is a credibility differentiator rather than a market one. The most widely used tools in this space — SwissADME [52], pkCSM [53] and ADMETlab [3] — are free, fixed-model web services that do not report on the TDC leaderboard at all, and they are popular precisely because they are convenient. Users are won on accessibility, not on leaderboard position. QSARena is designed to offer both, and we separate the two claims rather than conflating them.

Table 6: Positioning of QSARena relative to representative open-source, commercial and web-based molecular property-prediction tools. Performance entries report each tool’s **own published results** and are **not** head-to-head comparisons: rows use different splits, seeds and selection protocols, and “—” indicates no public TDC leaderboard results. Where a published claim could be re-scored against the full reference set we give both, because they differ substantially: ADMETboost’s 18 reported first places become 0 once all reference values are ranked together, and MaxQsaring’s 19 become 7. None of the tools in this table appears in the TDC leaderboard snapshots we captured, so every performance entry is publication-reported. ADMET-AI’s “highest average rank” is a 2024 self-reported aggregate, not per-endpoint supremacy. QSARena reuses ADMET-AI’s Chemprop-RDKit architecture through its **Chemprop v2** (D-MPNN + RDKit2D) variant but does not integrate ADMET-AI itself, and in this run that variant produced valid results on only 6 of 44 datasets, so parity with ADMET-AI is untested. “Code-free” means usable without scripting; “retrain” means the user can fit models on their own data; “suites” counts distinct benchmark collections the tool has been evaluated on in its own publications.

Tool	Access / licence	Code-free	Retrain	Architecture Suites breadth	Published TDC ADMET performance
QSARena (this work)	Open, MIT	Yes (Colab + CLI)	Yes	28 models, 7 families	5 Est. top-10 on 22/22 (test- selected); 15/22, median rank 8 under CV-only selection

Tool	Access / licence	Code-free	Retrain	Architecture breadth	Suites	Published TDC ADMET perform- ance
ADMET- AI [21]	Open, MIT	Partial (web + CLI)	Yes (training scripts)	1 (Chemprop- RDKit)	1 (41 datasets)	Highest average rank on TDC ADMET leaderboard (self- reported, 2024)
MaxQsaring [16]	Open, MIT	No (Python + CLI)	Yes	Multi- fingerprint feature selection + boosting	1 (22 datasets)	Reported first on 19/22; re-scored: 7 first places, top-3 on 19/22, median rank 2
ADMETboost [13]	Open	No (Python)	Yes	1 (XGBoost on multi- fingerprint features)	1 (22 datasets)	Reported first on 18/22 (2022); re-scored: 0 first places, top-3 on 6/22

Tool	Access / licence	Code-free	Retrain	Architecture Suites breadth		Published TDC ADMET performance
Schrödinger AutoQSAR / DeepAu- toQSAR [50, 51]	Commercial	Yes (Maestro GUI + CLI)	Yes	Multi- method ML + GNN	1	Top of three on 20/22, clearly best on 9/22 — a three-way vendor comparison vs ChemProp and Deep- Purpose, not a leaderboard rank —
QSARTuna [22]	Open, Apache-2.0	No (Python + CLI)	Yes	Representation × learners via Optuna; + Chemprop; UQ	9 (illustrative)	—
ZairaChem [23]	Open, GPL-3.0	No (CLI)	Yes	AutoML ensemble + distillation	1 (classification only)	State of the art out of the box on TDC classi- fication —
QSPRpred [24]	Open	No (Python library)	Yes	Flexible descriptors × learners	0 (framework)	—
DeepMol [25]	Open, Apache	No (Python)	Yes	AutoML over many models	1 (22 datasets)	Competitive pipelines across 22 TDC datasets

Tool	Access / licence	Code-free	Retrain	Architecture breadth	Suites	Published TDC ADMET performance
DeepPurpose [26]	Open	No (Python)	Yes	15 encoders, 50+ models	TDC loader	Used as TDC baselines
ChemXploreMD [28]	Open	Yes (desktop GUI)	Yes	Embeddings + tree ensembles	0 (physicochemical)	—
QSAR Workbench [49]	Commercial (Pipeline Pilot)	Yes (protocols)	Yes	Descriptor × learner grid	0 (2 CAESAR datasets)	—
ADMET Predictor (Simula- tions Plus)	Commercial	Yes (GUI + CLI)	Yes	Proprietary, ~100 endpoints	Proprietary	—
StarDrop / Auto- Modeller (Optib- rium)	Commercial	Yes (GUI)	Yes	PLS, RF, SVM, GP	Proprietary	—
SwissADME [52]	Free web	Yes (web)	No (fixed)	Fixed in-house models	n/a	—
pkCSM [53]	Free web	Yes (web)	No (fixed)	Graph- based signatures	n/a	—
ADMETlab 3.0 [3]	Free web	Yes (web)	No (fixed)	Multi-task graph attention	Internal	—

4 Limitations

Single split and single seed: the principal limitation of this work.

Every result reported here derives from one split and one random seed per dataset. The TDC convention is to report mean $\pm$ standard deviation over five independent seeds, and we do not meet it. Multi-seed evaluation over the official TDC splits is implemented in the runner (`--run-tdc22-multiseed-best`) and was enabled in this run’s configuration, but the stage executes only after all datasets finish and the run was interrupted before reaching it; repeating the full benchmark five times was in any case beyond the computational budget available for this study, since the single-seed run alone consumed 112 hours of A100 time (Section 3.7), implying roughly 560 hours for a five-seed replication.

The consequences should be stated plainly rather than minimized. We report no variance estimates, no confidence intervals and no significance tests, and we therefore cannot distinguish a genuine difference between two models from seed-to-seed noise. Concretely: (i) the per-dataset “winner” in Fig. 3, Additional file 1, Table S7 and Additional file 1 (Table S1) is the winner *on this split*, and an unknown fraction of the 44 winners would change under a different seed, so the win counts in Fig. 3 should be read as a coarse distribution across families rather than as a precise ranking; (ii) the family orderings in Table 2 are indicative, and the top three rows — ensembles at 75%, 3D pretrained at 70% and conventional machine learning at 66% within 5% of best — should be treated as a single indistinguishable group; (iii) the ensemble and fusion margins in Table 4 and Section 3.5 are small in absolute terms (median relative changes below 3%) and are the results most vulnerable to seed variance, so the conclusion we draw from them is directional rather than quantitative; and (iv) estimated leaderboard ranks (Section 3.4) inherit the same instability, which compounds the model-selection effect quantified there.

The hardware comparison in Section 3.10 gives an empirical sense of the noise floor: across 37 datasets with identical splits, changing the entire execution environment moved the best score by a median of 0.19%. Differences between models smaller than roughly a percent should not be interpreted.

What we believe survives this limitation is the leaderboard placement and the breadth of the winner distribution, because neither rests on close margins: top-ten placement on 35 of 37 comparable datasets (and 22 of 22 on the official TDC subset) clears the top-ten cutoff by a wide margin on most datasets; no family won more than 36% of datasets; the model-selection gap is large and one-sided; and the cost differences in Table 5 are orders of magnitude rather than percentages. Readers should treat every specific model-to-model comparison in this paper as provisional pending multi-seed replication, which we regard as the necessary next step for this work and the first thing any user of QSARena should run on a dataset that matters to them.

Model selection.

As quantified in Section 3.4, the test-selected headline is an optimistic maximum over 28 models. The cross-validation-selected protocol is the honest comparator but is itself

incomplete, since the Chemprop, Uni-Mol, MapLight + GNN and fusion families do not emit cross-validated metrics in this run.

Self-benchmarking asymmetry.

Competitors were not re-run under our pipeline; every comparison in Section 3.4 sets our single controlled run against other groups’ best published effort on the same dataset. The bias runs against us on tasks where a published entry reflects extensive tuning we did not perform, and in our favour wherever a published entry was itself selected on test data (see the reference-quality discussion in Section 3.4). We cannot quantify the net direction, and we do not claim the comparison is symmetric.

Estimated ranks are not submissions.

Ranks are computed against curated reference values of varying density (2 to 30 references per dataset) rather than by submitting to a live leaderboard, and 10 of the 37 comparable datasets use locally generated splits that are not leaderboard-equivalent.

Incomplete backend coverage.

Four of five Chemprop variants failed, TabPFN was disabled in this run, and the two larger Uni-Mol V2 variants produced no valid results, so Fig. 3 is not a fully balanced tournament (Section 3.8).

Two datasets incomplete.

`tdc_herg_central` was abandoned after more than 24 hours without completing, and `polaris_adme_fang_hppb_1` was interrupted when the run ended, though the latter had already produced a full model table and is retained.

Ensemble member filtering consults held-out data.

The `exclude_negative_test_r2_members` option, enabled in this run, drops ensemble members by held-out R^2 , and the correlated-member tie-break uses a held-out metric. Ensemble results are therefore mildly optimistic relative to a strictly blinded implementation; this is a configuration default we intend to change rather than a limitation of the method.

One target per task.

Multi-label datasets (Tox21, ToxCast) were reduced to a single label, so results on those datasets are not comparable to multi-task leaderboard entries.

Applicability domain and calibration.

The workspace includes applicability-domain utilities, but this benchmark does not report uncertainty calibration or domain-of-applicability diagnostics, which matter for regulatory use of any of these models.

5 Conclusions

Across 44 molecular property benchmarks from five suites evaluated under one leakage-controlled pipeline, QSARena’s best model per dataset reached estimated top-ten placement on 35 of 37 leaderboard-comparable datasets, including all 22 TDC datasets evaluated under official benchmark-group splits, with a median estimated rank of 3. This is achieved by an open-source pipeline that requires no bespoke architecture and no pretraining corpus.

No model family dominated. Ensembles over conventional learners won the most datasets (16 of 44) and were the most consistent; 3D pretrained models were a close second on both measures at roughly 55 times the median compute cost; conventional machine learning was third. Rich but compact molecular descriptors — the RDKit 2D panel and pharmacophore fingerprints — were selected far above their numerical share, while large hashed fingerprints were selected below theirs. Genetic-algorithm tuning was available but disabled throughout, so its value is untested here; the breadth of the fixed model library was sufficient on its own.

Two results bear on how such benchmarks should be read. First, top-ten placement falls from 35 of 37 to 25 of 37 when the model is chosen without consulting held-out data, and on the official TDC subset from 22 of 22 to 15 of 22 — but a matched-candidate-set control shows that 7 of those 10 placements are the value of a broad model library and only 3 are the cost of honest selection. Honest selection nonetheless removed every estimated first place. We regard the cross-validation-selected figure as the number practitioners should plan around, and we encourage the field to report both together with the candidate set each protocol saw, because a selection-protocol difference is easily mistaken for a model difference. Second, repeating the benchmark on a consumer laptop GPU changed the best score by a median of 0.19% across datasets with identical splits: the accuracy reported here does not depend on datacentre hardware, which bought throughput and backend coverage instead.

These conclusions rest on a single split and seed per dataset, a limitation imposed by the computational cost of the benchmark and discussed in full in Section 4. The claims we advance are accordingly those that survive that limitation — the leaderboard placement, the breadth of the winner distribution, the one-sided model-selection effect and the order-of-magnitude cost differences — rather than any close model-to-model comparison. Multi-seed replication remains the necessary next step.

A fourth result is about effort rather than accuracy. Every number above was produced under one fixed configuration, applied unchanged to all 44 datasets with no per-dataset feature engineering, architecture choice or hyperparameter tuning, and with evolutionary search disabled. Near-state-of-the-art placement is therefore what a single run yields, not the endpoint of a tuning campaign — and it is reachable from a browser, since the code-free notebook entry point runs end-to-end in Google Colab with no installation and no local hardware (Section 3.11). For domains such as regulatory toxicology, where the binding constraint is often expertise and infrastructure rather than algorithmic novelty, we regard this as the most consequential property of the tool.

QSARena delivers these results through a code-free notebook and a resume-safe command-line runner that share one workflow core, run without specialized hardware,

and emit the complete artifact trail needed to verify every number reported here. For practitioners who can run only one configuration, our results support a specific default: a MapLight-style descriptor panel with gradient-boosted trees, ensembled by inverse-error averaging, adding a 3D pretrained model where the compute budget allows and the endpoint plausibly depends on molecular shape.

Availability and requirements

Project name: QSARena

Project home page: <https://github.com/ScottCoffin/QSARena>

Operating system(s): Platform independent (Windows, macOS, Linux, Google Colab)

Programming language: Python 3.11

Installation: `pip install qsarena` (console scripts `qsarena-benchmark` and `qsarena-applicability-domain`), or clone the repository and run the scripts directly; the code-free notebook requires no installation at all and runs in Google Colab

Other requirements: RDKit, scikit-learn, pandas, NumPy, SciPy, joblib; heavy backends are optional extras (`qsarena[boosting]`, `[deep]`, `[graph]`, `[foundation]`, `[notebook]`). Exact versions for reproducing the benchmark are pinned in `requirements-cpu.txt` / `requirements-cuda.txt` and the conda environment files, which should be used in preference to the pip extras for exact reproduction

License: MIT License (OSI-approved); see the `LICENSE` file in the repository

Any restrictions to use by non-academics: None beyond the repository license

Abbreviations

ADMET: absorption, distribution, metabolism, excretion and toxicity; ADME: absorption, distribution, metabolism and excretion; AUPRC: area under the precision-recall curve; AUROC: area under the receiver operating characteristic curve; AutoML: automated machine learning; CFA: combinatorial fusion analysis; CV: cross-validation; D-MPNN: directed message-passing neural network; ECFP: extended-connectivity fingerprint; ErG: extended reduced graph; GIN: graph isomorphism network; GNN: graph neural network; MAE: mean absolute error; MCC: Matthews correlation coefficient; OOF: out-of-fold; POD: point of departure; QSAR: quantitative structure-activity relationship; RMSE: root-mean-square error; SMILES: simplified molecular-input line-entry system; TDC: Therapeutics Data Commons.

Supplementary information

Additional file 1. Supplementary tables S1–S4: per-dataset winning models with cross-validation-selected comparators (S1), staged component ablation (S2), feature-family selection summary (S3), and model coverage (S4).

Declarations

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Availability of data and materials. The QSARena source code, benchmark runner, analysis notebook and the complete benchmark artifacts analysed in this study are available in the QSARena repository, <https://github.com/ScottCoffin/QSARena>. The repository contains the workflow core (`qsar_workflow.core.py`), the notebook builder, the command-line benchmark runner (`run_qsarena.ga.benchmarks.py`), the dataset registry, the curated leaderboard reference tables, the per-dataset benchmark artifacts for the canonical A100 run (`benchmark_results/autoqsar.benchmark-20260623.153839/`: metrics, selected features, selector coefficients, split-signature hashes, runtime records, fusion candidate tables and ensemble weights) together with the consumer-GPU comparison run (`benchmark_results/benchmark_name-date/`), the analysis notebook (`portable_colab_qsar_bundle/benchmark_results-summary.ipynb`), and the scripts that regenerate every figure, table and quoted number in this paper (`render_manuscript_assets.py`, `verify_manuscript_numbers.py`). Pinned conda and pip environment specifications, an Apptainer container definition and Slurm/OpenStack submission scripts are included for independent re-execution.

All benchmark datasets are publicly available: the Therapeutics Data Commons via PyTDC (<https://tdcommons.ai>), MoleculeNet (<https://moleculenet.org>), Polaris (<https://polarishub.io>), the PODUAM point-of-departure datasets (<https://github.com/kejbo/PODUAM>) [31], and the ChemML bundled examples.

[**AUTHOR**] A versioned release archived on Zenodo, including the per-molecule prediction files excluded from the repository for size, will be deposited before publication and its DOI cited here.

Competing interests. The author declares no competing interests.

Funding. This work was supported by the California Office of Environmental Health Hazard Assessment (OEHHA). The funder had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

Authors’ contributions. SC conceived the study, developed the software, designed and executed the benchmark, analysed the results and wrote the manuscript. The author read and approved the final manuscript.

Acknowledgements. This work used Jetstream2 GPU at Indiana University through allocation CIS261142 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by U.S. National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296 [47]. Jetstream2 [46] is supported by the National Science Foundation under Grant 2005506. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author and do not necessarily reflect the views of the National Science Foundation.

The views expressed are those of the author and do not necessarily represent those of the California Environmental Protection Agency or the Office of Environmental Health Hazard Assessment. [**AUTHOR: confirm required agency disclaimer wording.**]

References

- [1] Sun D, Gao W, Hu H, Zhou S. Why 90% of clinical drug development fails and how to improve it? *Acta Pharmaceutica Sinica B*. 2022;12(7):3049–3062. <https://doi.org/10.1016/j.apsb.2022.02.002>.
- [2] Kola I, Landis J. Can the pharmaceutical industry reduce attrition rates? *Nature Reviews Drug Discovery*. 2004;3(8):711–715. <https://doi.org/10.1038/nrd1470>.
- [3] Fu L, Shi S, Yi J, Wang N, He Y, Wu Z, et al. ADMETlab 3.0: an updated comprehensive online ADMET prediction platform enhanced with broader coverage, improved performance, API functionality and decision support. *Nucleic Acids Research*. 2024;52(W1):W422–W431. <https://doi.org/10.1093/nar/gkae236>.
- [4] Komura H, Watanabe R, Mizuguchi K. The trends and future prospective of in silico models from the viewpoint of ADME evaluation in drug discovery. *Pharmaceutics*. 2023;15(11):2619. <https://doi.org/10.3390/pharmaceutics15112619>.
- [5] Wu Z, Ramsundar B, Feinberg EN, Gomes J, Geniesse C, Pappu AS, et al. MoleculeNet: a benchmark for molecular machine learning. *Chemical Science*. 2018;9(2):513–530. <https://doi.org/10.1039/C7SC02664A>.
- [6] Huang K, Fu T, Gao W, Zhao Y, Roohani Y, Leskovec J, et al. Therapeutics Data Commons: machine learning datasets and tasks for drug discovery and development. In: *Proceedings of the NeurIPS Datasets and Benchmarks Track*; 2021. ArXiv:2102.09548.
- [7] Méndez-Lucio O, Nicolaou CA, Earnshaw B. MolE: a foundation model for molecular graphs using disentangled attention. *Nature Communications*. 2024;15:9431. <https://doi.org/10.1038/s41467-024-53751-y>.
- [8] Sypetkowski M, Wenkel F, Poursafaei F, Dickson N, Suri K, Fradkin P, et al. On the scalability of GNNs for molecular graphs. In: *Advances in Neural Information Processing Systems* 37; 2024. .
- [9] Kläser K, Banaszewski B, Maddrell-Mander S, McLean C, Müller L, Parviz A, et al.: MiniMol: a parameter-efficient foundation model for molecular learning. ArXiv:2404.14986.
- [10] Zhou G, Gao Z, Ding Q, Zheng H, Xu H, Wei Z, et al. Uni-Mol: a universal 3D molecular representation learning framework. In: *International Conference on Learning Representations (ICLR)*; 2023. .
- [11] Xia J, Zhang L, Zhu X, Liu Y, Gao Z, Hu B, et al. Understanding the limitations of deep models for molecular property prediction: insights and solutions. In: *Advances in Neural Information Processing Systems* 36; 2023. .

- [12] Rodriguez-Perez R, Miljkovic F, Bajorath J. Machine learning in chemoinformatics and medicinal chemistry. *Annual Review of Biomedical Data Science*. 2022;5:43–65. <https://doi.org/10.1146/annurev-biodatasci-122120-124216>.
- [13] Tian H, Ketkar R, Tao P. ADMETboost: a web server for accurate ADMET prediction. *Journal of Molecular Modeling*. 2022;28:408. <https://doi.org/10.1007/s00894-022-05373-8>.
- [14] Notwell JH, Wood MW.: ADMET property prediction through combinations of molecular fingerprints. *ArXiv:2310.00174*.
- [15] Le HV, Ren W, Kim J, Yun Y, Park YB, Kim YJ, et al.: CaliciBoost: performance-driven evaluation of molecular representations for Caco-2 permeability prediction. *ArXiv:2506.08059*.
- [16] Xu C, Xu Y, Hu Z, Zhao X, Xie W, Chen W, et al. Unveiling optimal molecular features for hERG insights with automatic machine learning. *Journal of Pharmaceutical Analysis*. 2025;15(12):101411. <https://doi.org/10.1016/j.jpha.2025.101411>.
- [17] Kamuntavičius G, Paquet T, Bastas O, Šalkauskas D, Prat A, Abdel Aty H, et al. Benchmarking ML in ADMET predictions: the practical impact of feature representations in ligand-based models. *Journal of Cheminformatics*. 2025;17:108. <https://doi.org/10.1186/s13321-025-01041-0>.
- [18] Kapoor S, Narayanan A. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*. 2023;4(9):100804. <https://doi.org/10.1016/j.patter.2023.100804>.
- [19] Belfield SJ, Cronin MTD, Enoch SJ, Firman JW. Guidance for good practice in the application of machine learning in development of toxicological quantitative structure-activity relationships (QSARs). *PLOS ONE*. 2023;18(5):e0282924. <https://doi.org/10.1371/journal.pone.0282924>.
- [20] Koleiev I, Stratiichuk R, Shevchuk N, Melnychenko M, Nyporko O, Todoryshyn D, et al. An end-user audit of reproducibility, data leakage, and overfitting of the top-ranked ADMET prediction models in TDC leaderboards. *Journal of Chemical Information and Modeling*. 2026;<https://doi.org/10.1021/acs.jcim.6c00819>.
- [21] Swanson K, Walther P, Leitz J, Mukherjee S, Wu JC, Shivnaraine RV, et al. ADMET-AI: a machine learning ADMET platform for evaluation of large-scale chemical libraries. *Bioinformatics*. 2024;40(7):btae416. <https://doi.org/10.1093/bioinformatics/btae416>.
- [22] Mervin L, Voronov A, Kabeshov M, Engkvist O. QSARtuna: an automated QSAR modeling platform for molecular property prediction in drug design. *Journal of Chemical Information and Modeling*. 2024;64(14):5365–5374. <https://doi.org/10.1021/acs.jcim.4c00819>.

- [23] Turon G, Hlozek J, Woodland JG, Kumar A, Chibale K, Duran-Frigola M. First fully-automated AI/ML virtual screening cascade implemented at a drug discovery centre in Africa. *Nature Communications*. 2023;14:5736. <https://doi.org/10.1038/s41467-023-41512-2>.
- [24] van den Maagdenberg HW, Šícho M, Alencar Araripe D, Luukkonen S, Schoenmaker L, Jespers M, et al. QSPRpred: a flexible open-source quantitative structure-property relationship modelling tool. *Journal of Cheminformatics*. 2024;16(1):128. <https://doi.org/10.1186/s13321-024-00908-y>.
- [25] Correia J, Capela J, Rocha M. DeepMol: an automated machine and deep learning framework for computational chemistry. *Journal of Cheminformatics*. 2024;16(1):136. <https://doi.org/10.1186/s13321-024-00937-7>.
- [26] Huang K, Fu T, Glass LM, Zitnik M, Xiao C, Sun J. DeepPurpose: a deep learning library for drug-target interaction prediction. *Bioinformatics*. 2020;36(22-23):5545–5547. <https://doi.org/10.1093/bioinformatics/btaa1005>.
- [27] de Sá AGC, Ascher DB.: Auto-ADMET: an effective and interpretable AutoML method for chemical ADMET property prediction. *ArXiv:2502.16378*.
- [28] Marimuthu AN, McGuire BA. Machine learning pipeline for molecular property prediction using ChemXploreML. *Journal of Chemical Information and Modeling*. 2025;65(11):5424–5437. <https://doi.org/10.1021/acs.jcim.5c00516>.
- [29] Delaney JS. ESOL: estimating aqueous solubility directly from molecular structure. *Journal of Chemical Information and Computer Sciences*. 2004;44(3):1000–1005. <https://doi.org/10.1021/ci034243x>.
- [30] Mobley DL, Guthrie JP. FreeSolv: a database of experimental and calculated hydration free energies, with input files. *Journal of Computer-Aided Molecular Design*. 2014;28(7):711–720. <https://doi.org/10.1007/s10822-014-9747-x>.
- [31] von Borries K, Beckwith KV, Goodman JM, Chiu WA, Jolliet O, Fantke P. Uncertainty-aware machine learning to predict non-cancer human toxicity for the global chemicals market. *Nature Communications*. 2026;17:647. <https://doi.org/10.1038/s41467-025-67374-4>.
- [32] Landrum G.: RDKit: open-source cheminformatics. Accessed 2026-09-24. <https://www.rdkit.org>.
- [33] Rogers D, Hahn M. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*. 2010;50(5):742–754. <https://doi.org/10.1021/ci100050t>.

- [34] Durant JL, Leland BA, Henry DR, Nourse JG. Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*. 2002;42(6):1273–1280. <https://doi.org/10.1021/ci010132r>.
- [35] Gedeck P, Rohde B, Bartels C. QSAR – how good is it in practice? Comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*. 2006;46(5):1924–1936. <https://doi.org/10.1021/ci050413p>.
- [36] Stiefl N, Watson IA, Baumann K, Zaliani A. ErG: 2D pharmacophore descriptions for scaffold hopping. *Journal of Chemical Information and Modeling*. 2006;46(1):208–220. <https://doi.org/10.1021/ci050457y>.
- [37] Bemis GW, Mureko MA. The properties of known drugs. 1. Molecular frameworks. *Journal of Medicinal Chemistry*. 1996;39(15):2887–2893. <https://doi.org/10.1021/jm9602928>.
- [38] Pedregosa F, Varoquaux G, Gramfort A, Michel V, Thirion B, Grisel O, et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*. 2011;12:2825–2830.
- [39] Chen T, Guestrin C. XGBoost: a scalable tree boosting system. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; 2016. p. 785–794.
- [40] Ke G, Meng Q, Finley T, Wang T, Chen W, Ma W, et al. LightGBM: a highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems* 30; 2017. .
- [41] Prokhorenkova L, Gusev G, Vorobev A, Dorogush AV, Gulin A. CatBoost: unbiased boosting with categorical features. In: *Advances in Neural Information Processing Systems* 31; 2018. .
- [42] Yang K, Swanson K, Jin W, Coley C, Eiden P, Gao H, et al. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*. 2019;59(8):3370–3388. <https://doi.org/10.1021/acs.jcim.9b00237>.
- [43] Heid E, Greenman KP, Chung Y, Li SC, Graff DE, Vermeire FH, et al. Chemprop: a machine learning package for chemical property prediction. *Journal of Chemical Information and Modeling*. 2024;64(1):9–17. <https://doi.org/10.1021/acs.jcim.3c01250>.
- [44] Hollmann N, Müller S, Eggensperger K, Hutter F. TabPFN: a transformer that solves small tabular classification problems in a second. In: *International Conference on Learning Representations (ICLR)*; 2023. ArXiv:2207.01848.

- [45] Hsu DF, Chung YS, Kristal BS. Combinatorial fusion analysis: methods and practices of combining multiple scoring systems. In: Hsu HH, editor. *Advanced Data Mining Technologies in Bioinformatics*. IGI Global; 2006. p. 32–62.
- [46] Hancock DY, Fischer J, Lowe JM, Snapp-Childs W, Pierce M, Marru S, et al. Jetstream2: accelerating cloud computing via Jetstream. In: *Practice and Experience in Advanced Research Computing (PEARC '21)*. New York, NY, USA: ACM; 2021. .
- [47] Boerner TJ, Deems S, Furlani TR, Knuth SL, Towns J. ACCESS: Advancing Innovation: NSF’s Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support. In: *Practice and Experience in Advanced Research Computing (PEARC '23)*. New York, NY, USA: ACM; 2023. .
- [48] Deng J, Yang Z, Wang H, Ojima I, Samaras D, Wang F. A systematic study of key elements underlying molecular property prediction. *Nature Communications*. 2023;14(1):6395. <https://doi.org/10.1038/s41467-023-41948-6>.
- [49] Cox R, Green DVS, Luscombe CN, Malcolm N, Pickett SD. QSAR workbench: automating QSAR modeling to drive compound design. *Journal of Computer-Aided Molecular Design*. 2013;27(4):321–336. <https://doi.org/10.1007/s10822-013-9648-4>.
- [50] Dixon SL, Duan J, Smith E, Von Bargen CD, Sherman W, Repasky MP. AutoQSAR: an automated machine learning tool for best-practice quantitative structure-activity relationship modeling. *Future Medicinal Chemistry*. 2016;8(15):1825–1839. <https://doi.org/10.4155/fmc-2016-0093>.
- [51] Kaplan Z, Ehrlich S, Leswing K.: Benchmark study of DeepAutoQSAR, ChemProp and DeepPurpose on the ADMET subset of the Therapeutics Data Commons. Schrödinger, Inc. white paper. Accessed 2026-09-25. <https://www.schrodinger.com/life-science/learn/white-papers/benchmark-study-deepautoqsar-chemprop-and-deeppurpose-admet-subset-therapeutic-data/>.
- [52] Daina A, Michielin O, Zoete V. SwissADME: a free web tool to evaluate pharmacokinetics, drug-likeness and medicinal chemistry friendliness of small molecules. *Scientific Reports*. 2017;7:42717. <https://doi.org/10.1038/srep42717>.
- [53] Pires DEV, Blundell TL, Ascher DB. pkCSM: predicting small-molecule pharmacokinetic and toxicity properties using graph-based signatures. *Journal of Medicinal Chemistry*. 2015;58(9):4066–4072. <https://doi.org/10.1021/acs.jmedchem.5b00104>.